
Bases de datos NoSQL y su explotación: Cassandra y Spark



ASIGNATURA: BASES DE DATOS 2

PRÁCTICA 5

CURSO: 2025/26

COORDINADORA: Vázquez Martín, Lucía (NIP 871886)

Zhou Chen, Luna (NIP 812103)

Índice

Índice.....	2
1. Introducción.....	3
2. Entorno de trabajo y preparación.....	3
2.1. Organización del directorio de trabajo.....	3
2.2. Despliegue del entorno con Docker Compose.....	4
2.3. Acceso a Cassandra y Spark.....	5
2.4. Descripción de los datos utilizados.....	8
3. Explotación de Cassandra con CQL.....	8
3.1. Pregunta 1. Keyspace, equivalencia con el modelo relacional y replication_factor.....	8
3.2. Pregunta 2. Partition key, clustering key y efecto de CLUSTERING ORDER BY.....	9
3.3. Pregunta 3. Carga del CSV con COPY.....	10
3.4. Pregunta 4. Número de pruebas distintas y códigos.....	10
3.5. Pregunta 5. Resultado de SELECT DISTINCT.....	11
3.6. Pregunta 6. Resultado de COUNT(*).....	12
3.7. Pregunta 7. Consultas CQL solicitadas y limitaciones observadas.....	13
4. Explotación de Cassandra con Spark.....	17
4.1. Pregunta 8. Diferencia entre filas importadas y laboratory_table.count.....	17
4.2. Pregunta 9. Reescritura de las consultas de la Pregunta 7 con filter, select y avg.....	19
4.3. Pregunta 10. Análisis de pivotDF.....	20
4.4. Pregunta 11. Media para la prueba 1742-6 con select.....	21
5. Spark SQL.....	22
5.1. Pregunta 12. Resolución en Spark SQL de las consultas pedidas.....	22
5.2. Pregunta 13. Dos consultas adicionales no realizables con CQL.....	24
6. Dificultades encontradas y lecciones aprendidas.....	25
6.1. Problema con la imagen de Spark en el fichero docker-compose.yml.....	25
6.2. Incidencia en el acceso a Spark y resolución de dependencias de Cassandra.....	27
7. Conclusiones e impresiones personales.....	31
8. Distribución de trabajo.....	32
9. Bibliografía.....	33
10. Anexos.....	34
10.1. Anexo I. Archivos de configuración y scripts.....	34
10.1.1. docker-compose.yml (ORIGINAL).....	34
10.1.2. docker-compose.yml (ADAPTADO con imágenes Docker actualizadas).....	35
10.1.3. docker-compose.yml (ADAPTADO con bitnamilegacy).....	36
10.1.4. laboratorio.cql.....	37
10.1.5. pivot.sc.....	38
10.2. Anexo II. Salidas completas de ejecución y comprobación.....	39
10.2.1. Salida completa de ejecución de spark-shell con el conector de Cassandra.....	39
10.2.2. Salida completa de carga del script pivot.sc en Spark.....	41
10.2.3. Salida completa de laboratory_table.count.....	45

1. Introducción

La presente práctica, correspondiente a la **práctica 5**, tiene como objetivo abordar el trabajo con **bases de datos NoSQL** mediante el uso de **Apache Cassandra** y su explotación con **Spark**. En particular, se pretende **explorar las características básicas** de Cassandra como sistema gestor de bases de datos NoSQL y analizar cómo Spark puede emplearse como herramienta complementaria para la consulta y explotación de la información almacenada.

El trabajo se estructura en **varias preguntas** que permiten recorrer progresivamente las distintas fases de la práctica. En una **primera parte**, se examina el uso de **Cassandra mediante CQL**, abordando **conceptos fundamentales** como los keyspaces, la definición de claves de partición y de clustering, la carga de datos desde un fichero CSV y las limitaciones que presenta este lenguaje a la hora de formular determinadas consultas.

En una **segunda parte**, se aborda la explotación de los mismos datos mediante **Spark**, utilizando tanto la **API de DataFrames** como **Spark SQL**. Este enfoque permite realizar consultas más expresivas y flexibles sobre la información del laboratorio, así como comparar de forma práctica las capacidades de CQL frente a las ofrecidas por Spark.

2. Entorno de trabajo y preparación

Para la realización de esta práctica se ha **reutilizado el entorno de trabajo** configurado en prácticas anteriores, basado en una máquina virtual **TurnKey Core** ejecutada sobre VirtualBox y administrada desde el sistema anfitrión Windows mediante conexión SSH.

En esta práctica, el entorno proporcionado en el enunciado se basa en un fichero **docker-compose.yml** preparado para levantar **una instancia de Cassandra, un nodo maestro de Spark y dos nodos worker**. Asimismo, el enunciado indica que el directorio local desde el que se lanza Docker Compose queda montado dentro de los contenedores en la ruta **/data/**, lo que facilita el acceso compartido a scripts y ficheros de datos.

2.1. Organización del directorio de trabajo

Con el objetivo de mantener una **estructura ordenada y reproducible**, se preparó un **directorio específico dentro de la VM**. En dicho directorio se almacenaron tanto los ficheros de configuración como los scripts y datos necesarios para el desarrollo del trabajo.

Los **elementos principales** del directorio de trabajo fueron los siguientes:

- **docker-compose.yml**: fichero de despliegue del entorno.
- **laboratorio.cql**: script con las sentencias CQL de la parte de Cassandra.
- **pivot.sc**: script Scala utilizado en Spark.
- **dataset.csv**: fichero de datos descargado y descomprimido.

De forma orientativa, el **proceso de preparación del directorio** puede describirse así:

1. Acceso a la máquina virtual mediante SSH.

```
PS C:\Users\guest> ssh -p 2222 root@localhost
```

2. Crear un directorio específico para la práctica.

```
root@core ~# mkdir bd2-practica5
```

3. Situar en dicho directorio.

```
root@core ~# cd bd2-practica5/
root@core ~/bd2-practica5#
```

4. Copiar o guardar en él los ficheros proporcionados o desarrollados durante la práctica.

```
PS C:\Users\guest> scp -P 2222 D:\BASES2\Practicas\practica5\*
root@localhost:/root/bd2-practica5/
dataset.csv                                100% 814MB 21.2MB/s 00:38
docker-compose.yml                        100% 1428 278.9KB/s 00:00
laboratorio.cql                           100% 381 124.0KB/s 00:00
pivot.sc                                  100% 771 188.2KB/s 00:00
practica5_nosql.pdf                       100% 270KB 8.8MB/s 00:00
```

5. Verificar que todos ellos quedan accesibles desde los contenedores a través de /data/.

```
root@core ~/bd2-practica5# ls -l
total 833436
-rw-r--r-- 1 root root 853139928 Apr 21 22:01 dataset.csv
-rw-r--r-- 1 root root      1428 Apr 21 22:01 docker-compose.yml
-rw-r--r-- 1 root root      381 Apr 21 22:01 laboratorio.cql
-rw-r--r-- 1 root root      771 Apr 21 22:01 pivot.sc
-rw-r--r-- 1 root root 276213 Apr 21 22:01 practica5_nosql.pdf
```

2.2. Despliegue del entorno con Docker Compose

Una vez preparado el directorio de trabajo, se procedió al **arranque del entorno** mediante Docker Compose. Según el enunciado, todos los servicios necesarios pueden levantarse ejecutando **docker-compose up** desde el directorio donde se encuentre el fichero YAML.

No obstante, antes de realizar el despliegue fue necesario **adaptar el fichero [docker-compose.yml](#)** (la justificación se encuentra en [apartado 7.1](#)), ya que la configuración original hacía referencia a una imagen de Spark que no podía resolverse correctamente. Por ello, se modificó dicho archivo para **emplear una configuración funcional** que mantuviera la misma estructura lógica requerida en la práctica: una instancia de Cassandra, un nodo maestro de Spark, dos nodos worker y el montaje del directorio local en /data.

El **procedimiento** seguido puede describirse de la siguiente manera:

1. Arranque del entorno con Docker Compose.

```
root@core ~/bd2-practica5# docker-compose up
[+] up 17/28
```

```

✓ Image apache/spark:4.0.2      Pulled      150.0s
✓ Image cassandra:4.1           Pulled      70.6ss
✓ Network bd2-practica5_default Created      0.5s
✓ Container cassandra            Created      0.7s
✓ Container spark                 Created      0.2s
✓ Container spark-worker-1        Created      0.5s
✓ Container spark-worker-2        Created      0.5s
Attaching to cassandra, spark, spark-worker-1, spark-worker-2
...

```

Sin embargo, esta forma de ejecución mantiene el proceso **en primer plano** y deja la terminal ocupada mostrando continuamente la salida de los contenedores. Aunque resulta útil para observar el arranque inicial, para el trabajo habitual se optó por emplear la **opción -d**, que permite ejecutar el entorno **en segundo plano** y continuar utilizando la terminal para el resto de comandos. Por ello, el despliegue definitivo se realizó mediante:

```
root@core ~/bd2-practica5# docker-compose up -d
```

2. Verificación de que los contenedores de Cassandra y Spark se encuentran en ejecución.

```

root@core ~/bd2-practica5# docker compose ps
NAME                IMAGE                COMMAND                SERVICE        CREATED
STATUS              PORTS
cassandra            cassandra:4.1        "docker-entrypoint.s...  cassandra      10
minutes ago         Up 10 minutes        7000-7001/tcp, 7199/tcp, 9160/tcp, 0.0.0.0:9042->9042/tcp,
[::]:9042->9042/tcp
spark                apache/spark:4.0.2    "/opt/entrypoint.sh ...  spark          10
minutes ago         Up 10 minutes        0.0.0.0:7077->7077/tcp, [::]:7077->7077/tcp,
0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
spark-worker-1       apache/spark:4.0.2    "/opt/entrypoint.sh ...  spark-worker-1 10
minutes ago         Up 10 minutes        0.0.0.0:8081->8081/tcp, [::]:8081->8081/tcp
spark-worker-2       apache/spark:4.0.2    "/opt/entrypoint.sh ...  spark-worker-2 10
minutes ago         Up 10 minutes        0.0.0.0:8082->8082/tcp, [::]:8082->8082/tcp

```

Tras el despliegue, conviene comprobar que el entorno ha arrancado correctamente. Para ello, las evidencias más útiles en la memoria suelen ser:

- la salida de `docker-compose up -d`,
- la salida de `docker ps`,
- y, si resulta necesario, una captura o fragmento de logs del arranque.

2.3. Acceso a Cassandra y Spark

Una vez desplegados los servicios, se procedió a verificar el acceso a Cassandra y a Spark desde los contenedores correspondientes. Esta fase tuvo como objetivo comprobar que el entorno permitía trabajar con las herramientas previstas en el enunciado y que los ficheros

situados en el directorio de trabajo de la máquina virtual quedaban accesibles desde los contenedores mediante la ruta /data. En el caso de Cassandra, el acceso se realiza mediante cqlsh, mientras que en el caso de Spark se utiliza spark-shell junto con el conector de Cassandra indicado en la práctica. Asimismo, el enunciado establece que el script laboratorio.cql debe ejecutarse desde cqlsh mediante source '/data/laboratorio.cql', y que el script pivot.sc debe cargarse desde Spark mediante :load /data/pivot.sc.

Los pasos principales realizados fueron los siguientes:

1. Acceso a Cassandra mediante cqlsh

```
root@core ~/bd2-practica5# docker-compose exec cassandra cqlsh
ATTENTION: All commands will be saved to history file: /root/.cassandra/cqlsh_history
This may include sensitive information such as passwords.
To disable history, use --disable-history or set 'disabled = true' in the [history]
section of cqlshrc.
See https://cassandra.apache.org/doc/latest/tools/cqlsh.html for more information.

Connected to cloudfra at 127.0.0.1:9042
[cqlsh 6.1.0 | Cassandra 4.1.11 | CQL spec 3.4.6 | Native protocol v5]
Use HELP for help.

cqlsh>
```

La conexión se estableció correctamente, obteniéndose una sesión interactiva de Cassandra.

2. Ejecución del script CQL de preparación

```
cqlsh> source '/data/laboratorio.cql'
```

Esta ejecución permitió crear correctamente el keyspace laboratorio y la tabla hospital. No obstante, en este punto la tabla todavía no contenía datos, ya que la carga del fichero dataset.csv mediante la instrucción COPY forma parte de la Pregunta 3 y aún no se había implementado en el script.

3. Acceso a Spark con el conector de Cassandra

En relación con Spark, el acceso propuesto originalmente en el enunciado tuvo que adaptarse al entorno finalmente desplegado. Debido a la modificación del fichero docker-compose.yml y a la imagen de Spark utilizada, el ejecutable spark-shell no se encontraba accesible directamente desde el PATH del contenedor. Además, fue necesario indicar explícitamente la dirección del nodo maestro y una ruta válida para la caché de Ivy, con el fin de poder descargar el conector de Cassandra. Por este motivo, el acceso operativo a Spark se realizó finalmente mediante la siguiente secuencia.

```
root@core ~/bd2-practica5# docker-compose exec spark /opt/spark/bin/spark-shell \
--master spark://spark:7077 \
--conf spark.jars.ivy=/tmp/.ivy2 \
--packages com.datastax.spark:spark-cassandra-connector_2.12:3.3.0

Welcome to
```



```
Using Scala version 2.12.15 (OpenJDK 64-Bit Server VM, Java 11.0.20)
Type in expressions to have them evaluated.
Type :help for more information.

scala>
```

(La salida completa del proceso de recuento se incluye en el [Anexo 11.2.1](#))

En el caso de usar el fichero de configuración [docker-compose.yml](#) (modificado para usar bitnamilegacy) se puede ejecutar directamente este comando:

```
docker-compose exec spark spark-shell --packages
com.datastax.spark:spark-cassandra-connector_2.12:3.3.0
```

4. Carga del script Scala en Spark

```
scala> :load /data/pivot.sc
Loading /data/pivot.sc...
cassandraProperties: scala.collection.immutable.Map[String,String] =
Map(spark.cassandra.connection.host -> cassandra, spark.cassandra.connection.port ->
9042, keyspace -> laboratorio)
laboratory_table: org.apache.spark.sql.DataFrame = [std_observable_cd: string, person:
int ... 2 more fields]
+-----+-----+-----+-----+
|std_observable_cd|person|obs_value_nm|std_observable_st|
+-----+-----+-----+-----+
+-----+-----+-----+-----+

pivotDF: org.apache.spark.sql.DataFrame = [person: int]      (4 + 2) / 6]
+-----+=====>      (3 + 2) / 6]
|person|
+-----+
+-----+

res2: Long = 0
```

5. Comprobación básica del DataFrame cargado

```
scala> laboratory_table.count
res3: Long = 0
```

Esta salida confirmó que el script pivot.sc se había cargado correctamente y que Spark podía conectarse con Cassandra y acceder a la tabla laboratorio.hospital. Sin embargo, el recuento devolvió 0 porque en esta fase la tabla todavía se encontraba vacía. Por tanto, esta comprobación no debe interpretarse aún como una explotación real del conjunto de datos, sino como una validación de conectividad entre Spark y Cassandra y de la correcta accesibilidad de los scripts montados en /data.

La carga efectiva del fichero CSV y la obtención de resultados no vacíos se abordaron posteriormente, al resolver la Pregunta 3 del enunciado.

2.4. Descripción de los datos utilizados

La práctica trabaja sobre el fichero **dataset.csv**, descargado desde el [enlace](#) proporcionado en el enunciado. Este fichero contiene **resultados de pruebas de laboratorio** de un hospital, en formato CSV con **separador ;**. Cada línea representa el resultado de una prueba realizada a una persona concreta.

Los campos del fichero son los siguientes:

- **person**: identificador numérico de la persona.
- **std_observable_cd**: código alfanumérico de la prueba realizada.
- **std_observable_st**: descripción textual breve de la prueba.
- **obs_value_nm**: valor numérico observado en la prueba.

El enunciado destaca además una **característica importante** del conjunto de datos: una misma persona puede haberse sometido varias veces a la misma prueba, por lo que pueden existir varias líneas con la misma combinación de persona y código de prueba. Este detalle resulta especialmente relevante para interpretar correctamente algunos comportamientos posteriores de Cassandra y Spark.

3. Explotación de Cassandra con CQL

3.1. Pregunta 1. Keyspace, equivalencia con el modelo relacional y replication_factor

Pregunta 1. Observa el código:

```
CREATE KEYSPACE IF NOT EXISTS laboratorio
WITH REPLICATION = { 'class': 'SimpleStrategy',
                     'replication_factor': '1'};
```

¿Qué es un keyspace en Cassandra? ¿A qué concepto lo podemos asimilar en una base de datos relacional? ¿Qué implicaciones tiene 'replication_factor': '1'?

Un **keyspace** es el contenedor principal de organización en Apache Cassandra. Dentro de él se agrupan las **tablas** y, además, se define la **configuración de replicación** de los datos en el clúster. Por tanto, no solo sirve para organizar la información, sino también para indicar **cómo se distribuyen y cuántas copias** se mantienen de cada dato.

En una **base de datos relacional**, el concepto más parecido a un keyspace es una **base de datos**, ya que ambos agrupan un conjunto de tablas relacionadas bajo una misma administración general.

El parámetro **'replication_factor': '1'** indica que de cada dato solo se almacena **una única copia** en todo el clúster. Esto implica que **no existe tolerancia a fallos**: si el nodo que contiene esa información deja de estar disponible, los datos no podrán consultarse y podrían incluso perderse. Por ello, esta configuración solo resulta adecuada para **entornos**

de prueba o desarrollo, mientras que en **entornos de producción** se suelen utilizar factores de replicación mayores para garantizar mayor disponibilidad y fiabilidad.

3.2. Pregunta 2. Partition key, clustering key y efecto de CLUSTERING ORDER BY

Pregunta 2. Observa el código:

```
CREATE TABLE laboratorio.hospital (  
    std_observable_cd text,  
    person int,  
    std_observable_st text,  
    obs_value_nm float,  
    PRIMARY KEY ((std_observable_cd), person)  
) WITH CLUSTERING ORDER BY (person ASC);
```

Investiga qué es una partition key y una clustering key en Cassandra y añade una breve explicación de estos conceptos en la respuesta a esta pregunta.

¿Qué atributos de la tabla son partition keys y clustering keys? Si no se hubiese añadido al final de la declaración WITH CLUSTERING ORDER BY (person ASC), ¿qué efectos habría tenido sobre la base de datos?

En Cassandra, la **partition key** es la parte de la clave primaria que determina en qué partición se almacenan los datos y cómo se distribuyen entre los nodos del clúster. Es, por tanto, el elemento que Cassandra utiliza para localizar las filas. Por su parte, la **clustering key** sirve para ordenar las filas dentro de una misma partición.

En la definición:

```
PRIMARY KEY ((std_observable_cd), person)
```

el atributo **std_observable_cd** es la **partition key**, ya que aparece entre paréntesis propios en primer lugar. El atributo **person** es la **clustering key**, porque aparece después de la clave de partición dentro de la clave primaria. Esto significa que los datos se agrupan por código de prueba (std_observable_cd) y, dentro de cada grupo, se ordenan por identificador de persona (person).

La cláusula:

```
WITH CLUSTERING ORDER BY (person ASC)
```

indica que ese **orden dentro de cada partición** será **ascendente según person**. Si no se hubiera añadido, el efecto práctico en este caso habría sido el mismo, ya que Cassandra ordena **de forma ascendente por defecto** las clustering keys. Por tanto, la base de datos habría seguido almacenando y ordenando los datos igual; la cláusula solo deja explícito ese criterio de ordenación.

3.3. Pregunta 3. Carga del CSV con COPY

Pregunta 3. Añade al fichero `laboratorio.cql` el código necesario para cargar los datos del fichero csv a la tabla `laboratorio.hospital`. Utiliza la instrucción `COPY` de CQL.

Antes de realizar la carga del fichero CSV, si la tabla `hospital` contenía datos de importación previos, era conveniente **vaciarla** para evitar inconsistencias en la información almacenada.

```
cqlsh> USE laboratorio;  
cqlsh:laboratorio> TRUNCATE hospital;
```

Una vez limpia la tabla, **se añadió** al fichero `laboratorio.cql` una **instrucción COPY**, propia de CQL, que permite realizar **importaciones desde un fichero externo** a través de `cqlsh`. Dado que el fichero de datos está disponible en el directorio compartido `/data` del contenedor y que sus campos están separados por `;`, la instrucción utilizada fue la siguiente:

```
COPY laboratorio.hospital (person, std_observable_cd, std_observable_st, obs_value_nm)  
FROM '/data/dataset.csv'  
WITH HEADER = TRUE AND DELIMITER = ';';
```

En esta sentencia se indican las **columnas en el mismo orden** en que aparecen en el fichero CSV. Además, se emplea `HEADER = TRUE` para **ignorar la primera fila** del fichero, que contiene los **nombres de las columnas**, y se especifica el **delimitador** `;`.

La importación se completó correctamente, obteniéndose la siguiente salida:

```
Processed: 11354000 rows; Rate: 9125 rows/s; Avg. rate: 11795 rows/s  
11354000 rows imported from 1 files in 0 day, 0 hour, 16 minutes, and 2.614 seconds (0  
skipped).
```

Durante el proceso se observaron algunas incidencias relacionadas con la **estabilidad de Cassandra**. Al tratarse de una **carga masiva** sobre un fichero de gran tamaño, se produjeron **cierres temporales de la conexión** (`ConnectionShutdown`). Por tanto, los problemas detectados no se debían a errores en la sintaxis de la instrucción `COPY`, sino a la carga de procesamiento que suponía la importación.

3.4. Pregunta 4. Número de pruebas distintas y códigos

Pregunta 4. Añade al fichero `laboratorio.cql` el código necesario para hallar el número total de pruebas diferentes que se realizan en el laboratorio y sus códigos.

Para hallar el **número total de pruebas diferentes** que se realizan en el laboratorio y obtener sus **códigos**, se utiliza el atributo `std_observable_cd`, que identifica cada tipo de prueba en la tabla `laboratorio.hospital`. Para ello, se añade al fichero la consulta:

```
SELECT DISTINCT std_observable_cd  
FROM laboratorio.hospital;
```

```
std_observable_cd
```

```
-----  
26464-8  
26484-6  
3173-2  
2951-2  
34714-6  
26515-7  
2160-0  
20570-8  
1968-7  
26449-9  
2823-3  
26444-0  
1988-5  
30428-7  
1920-8  
1742-6  
1975-2  
28539-5  
5902-2  
41652-9  
26453-1  
3255-7  
26474-7  
28540-3  
8124-0
```

```
(25 rows)
```

Por tanto, puede concluirse que en el laboratorio se realizan **25 pruebas diferentes**, cuyos códigos son los mostrados en la consulta anterior.

En Cassandra, el **número total de valores distintos** puede deducirse directamente de la propia salida de cqlsh, que **al final del resultado** indica el número de filas devueltas. En este caso, la línea final (25 rows) permite identificar el total de pruebas distintas. Esto resulta especialmente útil porque **CQL no ofrece el mismo soporte que SQL relacional** para operaciones como COUNT(DISTINCT ...).

3.5. Pregunta 5. Resultado de SELECT DISTINCT

Pregunta 5. Ejecuta en cqlsh:

```
cqlsh> select distinct std_observable_cd, std_observable_st  
from laboratorio.hospital;
```

¿Qué respuesta obtienes? Explica brevemente por qué.

Al ejecutar en cqlsh la consulta se obtiene la siguiente respuesta:

```
cqlsh> select distinct std_observable_cd, std_observable_st
```

```
from laboratorio.hospital;  
InvalidRequest: Error from server: code=2200 [Invalid query] message="SELECT DISTINCT  
queries must only request partition key columns and/or static columns (not  
std_observable_st)"
```

Por tanto, la consulta no se ejecuta correctamente, sino que Cassandra devuelve un **error de tipo InvalidRequest**.

Esto se debe a que en Cassandra las **consultas con DISTINCT** están mucho más **limitadas** que en los sistemas relacionales. En concreto, solo pueden aplicarse sobre columnas que formen parte de la partition key o sobre columnas estáticas. En esta tabla, **std_observable_cd** sí forma parte de la clave de partición, pero **std_observable_st** no, por lo que no puede incluirse en una consulta SELECT DISTINCT.

La razón de esta restricción está en la propia arquitectura distribuida de Cassandra. A diferencia de una base de datos relacional, Cassandra **no está diseñada para realizar escaneos globales costosos** sobre todas las filas de la tabla con el fin de eliminar duplicados entre columnas arbitrarias. Permitir una operación de este tipo obligaría al sistema a recorrer grandes volúmenes de datos distribuidos entre múltiples nodos, lo que sería incompatible con el **modelo de consultas eficientes** sobre el que se basa Cassandra.

En consecuencia, esta consulta pone de manifiesto una **limitación importante de CQL**: el diseño de la tabla debe ajustarse a las consultas que se quieren realizar, y las operaciones de eliminación de duplicados sobre columnas no clave no están soportadas de forma general.

3.6. Pregunta 6. Resultado de COUNT(*)

Pregunta 6. Ejecuta en cqlsh:

```
cqlsh> select count(*) from laboratorio.hospital;
```

¿Qué respuesta obtienes? Explica brevemente por qué.

Al ejecutar en cqlsh la consulta se obtiene la siguiente respuesta:

```
cqlsh> select count(*) from laboratorio.hospital;  
ReadTimeout: Error from server: code=1200 [Coordinator node timed out waiting for  
replica nodes' responses] message="Operation timed out - received only 0 responses."  
info={'consistency': 'ONE', 'required_responses': 1, 'received_responses': 0}
```

Por tanto, la consulta no llega a completarse correctamente, sino que Cassandra devuelve un **error de tipo ReadTimeout**.

Este comportamiento se debe a que **COUNT(*)** es una **operación muy costosa** en Cassandra. A diferencia de los sistemas relacionales, Cassandra **no mantiene un contador global de filas** actualizado. Para calcular el resultado de esta consulta, el sistema necesita **recorrer** todas las particiones de la tabla y contar sus filas, lo que implica procesar un volumen muy elevado de datos.

En este caso, la tabla contiene **millones de registros**, por lo que la operación requiere más tiempo del permitido por el sistema para responder. Como consecuencia, el **nodo coordinador** agota el tiempo de espera antes de recibir la respuesta necesaria y la consulta termina con un **timeout**.

Este resultado muestra una **limitación importante** de Cassandra: no está orientada a realizar agregaciones globales costosas sobre grandes volúmenes de datos, sino a resolver consultas eficientes basadas en claves concretas.

3.7. Pregunta 7. Consultas CQL solicitadas y limitaciones observadas

Pregunta 7. Añade al fichero `laboratorio.cql` consultas CQL para:

- hallar la media de los valores numéricos observados (atributo `obs_value_nm`) para la prueba de código 3255-7;
- hallar la media de los valores numéricos observados de todas las pruebas realizadas a la persona con identificador igual a 123498;
- todas las pruebas con valor numérico observado mayor o igual a 100.

Atentos a los mensajes de respuesta para las dos últimas consultas.

Para responder a esta pregunta, se añadieron al fichero **laboratorio.cql** las siguientes consultas CQL:

```
-- 1. Media de los valores para la prueba 3255-7
SELECT AVG(obs_value_nm)
FROM laboratorio.hospital
WHERE std_observable_cd = '3255-7';

-- 2. Media de los valores para la persona 123498
SELECT AVG(obs_value_nm)
FROM laboratorio.hospital
WHERE person = 123498 ALLOW FILTERING;

-- 3. Pruebas con valor numérico mayor o igual a 100
SELECT *
FROM laboratorio.hospital
WHERE obs_value_nm >= 100 ALLOW FILTERING;
```

La **primera consulta** se ejecutó correctamente:

```
cqlsh> SELECT AVG(obs_value_nm)
        FROM laboratorio.hospital
        WHERE std_observable_cd = '3255-7';

system.avg(obs_value_nm)
-----
590.40747
```

```
(1 rows)
```

Esto es posible porque **std_observable_cd** es la partition key de la tabla, por lo que Cassandra puede **localizar directamente** la partición correspondiente y calcular la media sobre sus filas.

En cambio, para la **segunda consulta**, al intentar **calcular la media** de los valores observados para la persona 123498 **sin forzar el filtrado**, Cassandra devolvió un error:

```
cqlsh> SELECT AVG(obs_value_nm)
        FROM laboratorio.hospital
        WHERE person = 123498;

InvalidRequest: Error from server: code=2200 [Invalid query] message="Cannot execute this query as it might involve data filtering and thus may have unpredictable performance. If you want to execute this query despite the performance unpredictability, use ALLOW FILTERING"
```

Al **añadir ALLOW FILTERING**, la consulta sí pudo ejecutarse:

```
cqlsh> SELECT AVG(obs_value_nm)
        FROM laboratorio.hospital
        WHERE person = 123498 ALLOW FILTERING;

system.avg(obs_value_nm)
-----
                68.96481

(1 rows)

Warnings :
Aggregation query used without partition key
```

El **resultado** obtenido fue **68.96481**, pero Cassandra advirtió que la agregación se estaba realizando sin utilizar la clave de partición. Esto implica que la **consulta no está optimizada** y que el sistema debe examinar un volumen mucho mayor de datos, con un rendimiento impredecible.

Algo similar ocurrió con la **tercera consulta**. Al intentar obtener todas las pruebas con valor numérico mayor o igual a 100 **sin ALLOW FILTERING**, Cassandra volvió a dar error:

```
cqlsh> SELECT *
        FROM laboratorio.hospital
        WHERE obs_value_nm >= 100;

InvalidRequest: Error from server: code=2200 [Invalid query] message="Cannot execute this query as it might involve data filtering and thus may have unpredictable performance. If you want to execute this query despite the performance unpredictability, use ALLOW FILTERING"
```

Solo al **añadir ALLOW FILTERING** fue posible ejecutarla:

```
cqlsh> SELECT *
        FROM laboratorio.hospital
        WHERE obs_value_nm >= 100 ALLOW FILTERING;
```

std_observable_cd	person	obs_value_nm	std_observable_st
26464-8	3865	155.60001	Leukocytes [# /volume] in Blood
26464-8	3866	155.60001	Leukocytes [# /volume] in Blood
26464-8	4840	199	Leukocytes [# /volume] in Blood
26464-8	5057	155.60001	Leukocytes [# /volume] in Blood
26464-8	5168	307.89999	Leukocytes [# /volume] in Blood
26464-8	5510	199	Leukocytes [# /volume] in Blood
26464-8	6224	156.2	Leukocytes [# /volume] in Blood
26464-8	6585	307.89999	Leukocytes [# /volume] in Blood
26464-8	6995	156.2	Leukocytes [# /volume] in Blood
26464-8	7703	307.89999	Leukocytes [# /volume] in Blood
26464-8	9510	177.10001	Leukocytes [# /volume] in Blood
26464-8	9623	137.87	Leukocytes [# /volume] in Blood
26464-8	10845	137.87	Leukocytes [# /volume] in Blood
26464-8	10983	155.60001	Leukocytes [# /volume] in Blood
26464-8	12891	156.2	Leukocytes [# /volume] in Blood
26464-8	12892	156.2	Leukocytes [# /volume] in Blood
26464-8	14030	307.89999	Leukocytes [# /volume] in Blood
26464-8	14495	307.89999	Leukocytes [# /volume] in Blood
26464-8	14946	137.87	Leukocytes [# /volume] in Blood
26464-8	15807	156.2	Leukocytes [# /volume] in Blood
26464-8	15856	137.87	Leukocytes [# /volume] in Blood
26464-8	23319	177	Leukocytes [# /volume] in Blood
26464-8	23682	177	Leukocytes [# /volume] in Blood
26464-8	24709	177	Leukocytes [# /volume] in Blood
26464-8	25898	177	Leukocytes [# /volume] in Blood
26464-8	25899	177	Leukocytes [# /volume] in Blood
26464-8	26785	199	Leukocytes [# /volume] in Blood
26464-8	27008	307.89999	Leukocytes [# /volume] in Blood
26464-8	27191	137.87	Leukocytes [# /volume] in Blood
26464-8	27213	156.2	Leukocytes [# /volume] in Blood
26464-8	28508	177	Leukocytes [# /volume] in Blood
26464-8	29706	177	Leukocytes [# /volume] in Blood
26464-8	30068	156.2	Leukocytes [# /volume] in Blood
26464-8	31009	155.60001	Leukocytes [# /volume] in Blood
26464-8	32234	199	Leukocytes [# /volume] in Blood
26464-8	32457	137.87	Leukocytes [# /volume] in Blood
26464-8	32488	156.2	Leukocytes [# /volume] in Blood
26464-8	36323	177	Leukocytes [# /volume] in Blood
26464-8	36359	155.60001	Leukocytes [# /volume] in Blood
26464-8	38045	155.60001	Leukocytes [# /volume] in Blood
26464-8	38836	155.60001	Leukocytes [# /volume] in Blood
26464-8	39176	177	Leukocytes [# /volume] in Blood
26464-8	40443	156.2	Leukocytes [# /volume] in Blood
26464-8	40444	156.2	Leukocytes [# /volume] in Blood
26464-8	41065	137.87	Leukocytes [# /volume] in Blood
26464-8	41531	156.2	Leukocytes [# /volume] in Blood
26464-8	41925	155.60001	Leukocytes [# /volume] in Blood
26464-8	42161	307.89999	Leukocytes [# /volume] in Blood
26464-8	42841	199	Leukocytes [# /volume] in Blood

26464-8	42842	199	Leukocytes	[#/volume]	in Blood
26464-8	43409	177	Leukocytes	[#/volume]	in Blood
26464-8	44871	307.89999	Leukocytes	[#/volume]	in Blood
26464-8	44872	307.89999	Leukocytes	[#/volume]	in Blood
26464-8	44892	155.60001	Leukocytes	[#/volume]	in Blood
26464-8	46624	137.87	Leukocytes	[#/volume]	in Blood
26464-8	46625	137.87	Leukocytes	[#/volume]	in Blood
26464-8	46842	177	Leukocytes	[#/volume]	in Blood
26464-8	48444	199	Leukocytes	[#/volume]	in Blood
26464-8	48655	307.89999	Leukocytes	[#/volume]	in Blood
26464-8	49097	156.2	Leukocytes	[#/volume]	in Blood
26464-8	49333	177	Leukocytes	[#/volume]	in Blood
26464-8	49894	177	Leukocytes	[#/volume]	in Blood
26464-8	49895	177	Leukocytes	[#/volume]	in Blood
26464-8	51788	156.2	Leukocytes	[#/volume]	in Blood
26464-8	52716	177	Leukocytes	[#/volume]	in Blood
26464-8	53563	199	Leukocytes	[#/volume]	in Blood
26464-8	53769	307.89999	Leukocytes	[#/volume]	in Blood
26464-8	53924	137.87	Leukocytes	[#/volume]	in Blood
26464-8	54003	156.2	Leukocytes	[#/volume]	in Blood
26464-8	55478	156.2	Leukocytes	[#/volume]	in Blood
26464-8	57011	307.89999	Leukocytes	[#/volume]	in Blood
26464-8	57728	307.89999	Leukocytes	[#/volume]	in Blood
26464-8	57805	155.60001	Leukocytes	[#/volume]	in Blood
26464-8	57806	155.60001	Leukocytes	[#/volume]	in Blood
26464-8	59904	137.87	Leukocytes	[#/volume]	in Blood
26464-8	60214	156.2	Leukocytes	[#/volume]	in Blood
26464-8	60330	199	Leukocytes	[#/volume]	in Blood
26464-8	60732	137.87	Leukocytes	[#/volume]	in Blood
26464-8	61945	155.60001	Leukocytes	[#/volume]	in Blood
26464-8	63234	177	Leukocytes	[#/volume]	in Blood
26464-8	65010	109	Leukocytes	[#/volume]	in Blood
26464-8	65106	155.60001	Leukocytes	[#/volume]	in Blood
26464-8	65852	156.2	Leukocytes	[#/volume]	in Blood
26464-8	65853	156.2	Leukocytes	[#/volume]	in Blood
26464-8	66268	155.60001	Leukocytes	[#/volume]	in Blood
26464-8	66269	155.60001	Leukocytes	[#/volume]	in Blood
26464-8	66270	155.60001	Leukocytes	[#/volume]	in Blood
26464-8	66729	155.60001	Leukocytes	[#/volume]	in Blood
26464-8	67358	177	Leukocytes	[#/volume]	in Blood
26464-8	68146	137.87	Leukocytes	[#/volume]	in Blood
26464-8	68264	199	Leukocytes	[#/volume]	in Blood
26464-8	68265	199	Leukocytes	[#/volume]	in Blood
26464-8	68369	199	Leukocytes	[#/volume]	in Blood
26464-8	68530	155.60001	Leukocytes	[#/volume]	in Blood
26464-8	68531	155.60001	Leukocytes	[#/volume]	in Blood
26464-8	69166	177.10001	Leukocytes	[#/volume]	in Blood
26464-8	69771	137.87	Leukocytes	[#/volume]	in Blood
26464-8	69772	137.87	Leukocytes	[#/volume]	in Blood
26464-8	69938	155.60001	Leukocytes	[#/volume]	in Blood
26464-8	70039	307.89999	Leukocytes	[#/volume]	in Blood
---MORE---					

La consulta devolvió un listado muy extenso de resultados, tan grande que su visualización en cqlsh resultó poco manejable y apareció repetidamente el aviso ---MORE---

En conjunto, estas pruebas muestran una **limitación importante** de Cassandra: solo resuelve de forma eficiente las consultas que están alineadas con la clave primaria definida en la tabla. Cuando se intenta **filtrar o agregar** sobre columnas que no forman parte de la clave de partición, el sistema exige ALLOW FILTERING y advierte que el **rendimiento puede ser impredecible**. Esto refleja que, en Cassandra, el modelo de datos debe diseñarse teniendo en cuenta de antemano las consultas que se quieren realizar.

4. Explotación de Cassandra con Spark

4.1. Pregunta 8. Diferencia entre filas importadas y `laboratory_table.count`

Pregunta 8. En la sección anterior, se han cargado los datos del fichero csv a la tabla de Cassandra. Al final de la carga, el SGBD informó de que se habían importado 11.354.000 de las (una por cada línea del fichero csv). Sin embargo, `laboratory_table.count` nos indica que la tabla tiene muchas menos filas. Explica la razón. Ten en cuenta la observación “Importante” que se hizo en la sección 2.2 e investiga sobre la política *Last-Write-Wins* (LWW) para resolver conflictos en Cassandra.

Al finalizar la **carga del fichero CSV en Cassandra**, el sistema informó de que se habían importado 11.354.000 filas, es decir, una por cada línea del fichero de entrada. Sin embargo, el **número de filas** que finalmente quedan almacenadas en la tabla **no coincide** con ese valor, sino que **es considerablemente menor**. La **razón principal** se encuentra en la definición de la clave primaria de la tabla:

```
PRIMARY KEY ((std_observable_cd), person)
```

Esto significa que **cada fila** queda identificada únicamente por la **combinación del código de prueba** (`std_observable_cd`) y el **identificador de persona** (`person`). No obstante, el propio enunciado indica que a una misma persona se le puede realizar la misma prueba varias veces. Por tanto, en el fichero CSV **existen múltiples líneas que comparten** exactamente esa **misma clave primaria**.

En Cassandra **no pueden coexistir** varias filas con la misma clave primaria. Cuando se insertan varios registros con la misma clave, el sistema aplica la **política Last-Write-Wins** (LWW), según la cual **solo se conserva la última escritura** asociada a esa clave y las anteriores quedan sobrescritas. Como consecuencia, aunque durante la importación se procesen 11.354.000 líneas, muchas de ellas **no permanecen como filas distintas** en la tabla final, ya que **reemplazan** a registros previos con la misma clave.

En nuestra ejecución, tras cargar el script **pivot.sc**, se comprobó que Spark accedía correctamente a la tabla **hospital** y generaba el DataFrame **laboratory_table**. A continuación, el script construía también el DataFrame **pivotDF**, cuyo recuento fue:

```
scala> :load /data/pivot.sc
Loading /data/pivot.sc...
```

```
cassandraProperties: scala.collection.immutable.Map[String,String] =
Map(spark.cassandra.connection.host -> cassandra, spark.cassandra.connection.port ->
9042, keyspace -> laboratorio)

laboratory_table: org.apache.spark.sql.DataFrame = [std_observable_cd: string, person:
int ... 2 more fields]

pivotDF: org.apache.spark.sql.DataFrame = [person: int, 1742-6: double ... 24 more
fields]

res2: Long = 306076
```

(La salida completa del proceso de carga se incluye en el [Anexo 11.2.2](#))

Este valor no corresponde al número de filas de la **tabla Cassandra**, sino al número de filas del **DataFrame pivotado**, es decir, al número de personas distintas tras agrupar por **person**. Aun así, ilustra que el volumen de datos con el que se trabaja después de las transformaciones **es inferior** al número de líneas procesadas en la importación.

A continuación, se intentó realizar el **recuento directo** de la tabla mediante:

```
scala> laboratory_table.count
com.datastax.oss.driver.api.core.servererrors.ReadFailureException: Cassandra failure
during read query at consistency LOCAL_ONE (1 responses were required but only 0 replica
responded, 1 failed)

org.apache.spark.SparkException: Job aborted due to stage failure
```

(La salida completa del proceso de recuento se incluye en el [Anexo 11.2.3](#))

La operación no llegó a completarse, sino que Spark devolvió un **error de tipo ReadFailureException**, indicando que Cassandra no pudo responder correctamente a la lectura global solicitada.

En consecuencia, en la ejecución **no se obtuvo el número exacto de filas** finales almacenadas en hospital, pero sí puede explicarse por qué ese número debe ser mucho menor que 11.354.000: la combinación de una clave primaria no preparada para distinguir **repeticiones temporales** y la **política Last-Write-Wins** provoca la **sobrescritura** de muchos registros durante la carga.

Por tanto, la diferencia entre el **número de filas importadas** y el **número real de filas almacenadas** no se debe a un error de carga, sino al propio modelo de datos definido en Cassandra. El sistema procesa todas las líneas del CSV, pero **solo conserva una fila** por cada combinación única de **std_observable_cd** y **person**, sobrescribiendo las anteriores cuando aparecen duplicados de esa clave.

4.2. Pregunta 9. Reescritura de las consultas de la Pregunta 7 con filter, select y avg

Pregunta 9. Investiga cómo se resuelven las consultas de la Pregunta 7 usando los métodos `filter`, `select` y la función de agregación `avg`. Puedes consultar ejemplos en <https://sparkbyexamples.com/spark/spark-sql-aggregate-functions/>. Añade las consultas al fichero `pivot.sc`

Para resolver en Spark las consultas planteadas en la Pregunta 7, se utilizaron los métodos **filter** y **select** sobre el DataFrame **laboratory_table**, junto con la función de agregación **avg**. Dado que esta función pertenece al **paquete de funciones** de Spark SQL, fue necesario añadir previamente al fichero **pivot.sc** la siguiente importación:

```
import org.apache.spark.sql.functions._
```

A continuación, se añadieron las **siguientes consultas**:

```
val media_prueba_3255_7 = laboratory_table
  .filter("std_observable_cd = '3255-7'")
  .select(avg("obs_value_nm"))
  .show()

val media_persona_123498 = laboratory_table
  .filter("person = 123498")
  .select(avg("obs_value_nm"))
  .show()

val pruebas_mayor_100 = laboratory_table
  .filter("obs_value_nm >= 100")
  .show()
```

La **primera consulta** filtra las filas cuyo código de prueba es 3255-7 y, sobre ese subconjunto, calcula la media de los valores del atributo `obs_value_nm`. Es, por tanto, la equivalente en Spark a la consulta pedida en Cassandra para obtener la media de una prueba concreta. La salida obtenida fue la siguiente:

```
+-----+
|avg(obs_value_nm)|
+-----+
|590.4074615851787|
+-----+

media_prueba_3255_7: Unit = ()
```

La **segunda consulta** filtra las filas correspondientes a la persona con identificador 123498 y calcula la media de todos los valores observados asociados a esa persona. De este modo, resuelve en Spark la consulta que en Cassandra resultaba problemática por no usar la clave de partición. El resultado obtenido fue:

```
+-----+
|avg(obs_value_nm)|
+-----+
|68.96480902250518|
+-----+

media_persona_123498: Unit = ()
```

La **tercera consulta** filtra todas las filas cuyo valor numérico observado es mayor o igual que 100. A diferencia de las dos anteriores, aquí no se calcula ninguna agregación, sino que simplemente se devuelven las filas que cumplen la condición. Esta fue la salida obtenida:

```
+-----+-----+-----+-----+
|std_observable_cd|person|obs_value_nm|std_observable_st|
+-----+-----+-----+-----+
|3255-7|1|316.0|Fibrinogen [Mass/...|
|3255-7|2|316.0|Fibrinogen [Mass/...|
|3255-7|4|489.0|Fibrinogen [Mass/...|
|3255-7|5|303.0|Fibrinogen [Mass/...|
|3255-7|6|425.0|Fibrinogen [Mass/...|
|3255-7|9|528.0|Fibrinogen [Mass/...|
|3255-7|11|564.0|Fibrinogen [Mass/...|
|3255-7|16|531.0|Fibrinogen [Mass/...|
|3255-7|17|564.0|Fibrinogen [Mass/...|
|3255-7|20|388.0|Fibrinogen [Mass/...|
|3255-7|21|564.0|Fibrinogen [Mass/...|
|3255-7|22|531.0|Fibrinogen [Mass/...|
|3255-7|23|516.0|Fibrinogen [Mass/...|
|3255-7|24|538.0|Fibrinogen [Mass/...|
|3255-7|26|649.0|Fibrinogen [Mass/...|
|3255-7|27|703.0|Fibrinogen [Mass/...|
|3255-7|28|498.0|Fibrinogen [Mass/...|
|3255-7|29|703.0|Fibrinogen [Mass/...|
|3255-7|30|303.0|Fibrinogen [Mass/...|
|3255-7|32|551.0|Fibrinogen [Mass/...|
+-----+-----+-----+-----+

only showing top 20 rows

pruebas_mayor_100: Unit = ()
```

Estas consultas muestran que Spark permite **resolver con facilidad** operaciones de filtrado y agregación sobre cualquier columna del DataFrame, sin las restricciones que imponía Cassandra en la Pregunta 7.

4.3. Pregunta 10. Análisis de pivotDF

Pregunta 10. Explica qué tarea realiza la instrucción del fichero pivot.sc

```
val pivotDF = laboratory_table.groupBy("person").
  pivot("std_observable_cd").
  sum("obs_value_nm")
```

¿Qué columnas tendrá el dataframe `pivotDF`? ¿Qué datos contiene?

La instrucción realiza una **transformación** de tipo pivot sobre el DataFrame **laboratory_table**, con el objetivo de reorganizar la información en un formato más adecuado para el análisis.

En primer lugar, **groupBy("person")** agrupa todas las filas por el identificador de la persona. Esto significa que, a partir de ese momento, cada grupo de datos queda asociado a una persona concreta. A continuación, **pivot("std_observable_cd")** toma los distintos valores del atributo **std_observable_cd**, es decir, los distintos códigos de prueba, y los convierte en columnas del nuevo DataFrame. En lugar de tener una fila distinta por cada prueba realizada, el resultado pasa a tener una única fila por persona y una columna para cada tipo de prueba. Por último, **sum("obs_value_nm")** calcula, para cada persona y para cada código de prueba, la suma de los valores observados en **obs_value_nm**. Es decir, si una misma persona tiene varios registros asociados a una determinada prueba, el valor que aparecerá en la celda correspondiente será la suma de todos esos valores.

Como resultado, el DataFrame **pivotDF** tendrá una estructura en la que **cada persona** ocupa una fila y **cada tipo de prueba** pasa a ocupar una columna.

- una **primera columna** llamada **person**, que identifica a cada persona;
- una **columna adicional** por cada código de prueba distinto existente en los datos, por ejemplo 1742-6, 1920-8, 3255-7, etc.

En cuanto a los **datos que contiene**:

- **cada fila** representa una persona distinta;
- **cada columna de prueba** contiene la suma de los valores **obs_value_nm** asociados a esa persona para dicha prueba;
- si una persona **no tiene registros** para una determinada prueba, en esa celda aparecerá **null**.

Por tanto, **pivotDF** no almacena ya los datos en formato de observaciones individuales, sino en una **tabla resumida por personas y tipos de prueba**. Esta reorganización facilita comparaciones entre pruebas y permite trabajar de forma más cómoda con análisis.

4.4. Pregunta 11. Media para la prueba 1742-6 con select

Pregunta 11. Añade el código necesario en el fichero `pivot.sc` para hallar la media de los valores observados en la prueba 1742-6 utilizando el método `select` de DataFrame.

Para hallar la **media de los valores** observados correspondientes a la prueba con código 1742-6, se añadió al fichero **pivot.sc** una consulta basada en los métodos **filter** y **select** del DataFrame **laboratory_table**, junto con la función de agregación **avg**. Dado que se utiliza dicha función, fue necesario disponer de la importación correspondiente de Spark SQL.

El código ejecutado fue el siguiente:

```
val media_prueba_1742_6 = laboratory_table
```

```
.filter("std_observable_cd = '1742-6'")  
.select(avg("obs_value_nm"))  
.show()
```

Esta consulta funciona del siguiente modo. En primer lugar, **filter("std_observable_cd = '1742-6'")** selecciona únicamente las filas asociadas a la prueba 1742-6. A continuación, **select(avg("obs_value_nm"))** calcula la media de los valores almacenados en el atributo `obs_value_nm` para ese subconjunto de filas. Finalmente, **show()** muestra el resultado por pantalla en la consola de Spark.

La salida obtenida fue la siguiente:

```
+-----+  
|avg(obs_value_nm)|  
+-----+  
|37.33964581318896|  
+-----+
```

Por tanto, la media de los valores observados para la prueba 1742-6 fue 37.33964581318896.

5. Spark SQL

5.1. Pregunta 12. Resolución en Spark SQL de las consultas pedidas

Pregunta 12. Utilizando Spark SQL, añade el código necesario en el fichero `pivot.sc` para realizar las consultas de la Pregunta 7 y Pregunta 11.

Para reproducir en Spark SQL las consultas planteadas en la Pregunta 7 y en la Pregunta 11, primero fue necesario registrar el DataFrame **laboratory_table** como una **vista temporal**. De este modo, los datos cargados desde Cassandra pudieron consultarse mediante sentencias SQL sobre dicha vista. La instrucción utilizada fue la siguiente:

```
laboratory_table.createOrReplaceTempView("lab")
```

Una vez creada la vista temporal **lab**, se añadieron al fichero **pivot.sc** las consultas:

```
val consulta1 = spark.sql("SELECT AVG(obs_value_nm) FROM lab WHERE std_observable_cd =  
'3255-7']").show()  
  
val consulta2 = spark.sql("SELECT AVG(obs_value_nm) FROM lab WHERE person =  
123498").show()  
  
val consulta3 = spark.sql("SELECT * FROM lab WHERE obs_value_nm >= 100").show()  
  
val consulta4 = spark.sql("SELECT AVG(obs_value_nm) FROM lab WHERE std_observable_cd =  
'1742-6']").show()
```

La **primera consulta** calcula la media de los valores observados para la prueba con código 3255-7. La salida obtenida fue:

```

+-----+
|avg(obs_value_nm)|
+-----+
|590.4074615851787|
+-----+

consulta1: Unit = ()

```

La **segunda consulta** calcula la media de los valores observados correspondientes a todas las pruebas realizadas a la persona con identificador 123498. El resultado obtenido fue:

```

+-----+
|avg(obs_value_nm)|
+-----+
|68.96480902250518|
+-----+

consulta2: Unit = ()

```

La **tercera consulta** recupera todas las filas cuyo valor numérico observado es mayor o igual que 100. Como el número de resultados es muy elevado, Spark solo mostró por pantalla las primeras filas:

```

+-----+-----+-----+-----+
|std_observable_cd|person|obs_value_nm|std_observable_st|
+-----+-----+-----+-----+
|3255-7|1|316.0|Fibrinogen [Mass/...|
|3255-7|2|316.0|Fibrinogen [Mass/...|
|3255-7|4|489.0|Fibrinogen [Mass/...|
|3255-7|5|303.0|Fibrinogen [Mass/...|
|3255-7|6|425.0|Fibrinogen [Mass/...|
|3255-7|9|528.0|Fibrinogen [Mass/...|
|3255-7|11|564.0|Fibrinogen [Mass/...|
|3255-7|16|531.0|Fibrinogen [Mass/...|
|3255-7|17|564.0|Fibrinogen [Mass/...|
|3255-7|20|388.0|Fibrinogen [Mass/...|
|3255-7|21|564.0|Fibrinogen [Mass/...|
|3255-7|22|531.0|Fibrinogen [Mass/...|
|3255-7|23|516.0|Fibrinogen [Mass/...|
|3255-7|24|538.0|Fibrinogen [Mass/...|
|3255-7|26|649.0|Fibrinogen [Mass/...|
|3255-7|27|703.0|Fibrinogen [Mass/...|
|3255-7|28|498.0|Fibrinogen [Mass/...|
|3255-7|29|703.0|Fibrinogen [Mass/...|
|3255-7|30|303.0|Fibrinogen [Mass/...|
|3255-7|32|551.0|Fibrinogen [Mass/...|
+-----+-----+-----+-----+

only showing top 20 rows

consulta3: Unit = ()

```

La **cuarta consulta** reproduce en Spark SQL la operación de la Pregunta 11, es decir, calcula la media de los valores observados para la prueba con código 1742-6. La salida fue:

```
+-----+
|avg(obs_value_nm)|
+-----+
|37.33964581318896|
+-----+

consulta4: Unit = ()
```

Estas ejecuciones muestran que Spark SQL permite expresar de forma directa, mediante **sentencias SQL convencionales**, consultas que en CQL resultaban más limitadas o requerían ALLOW FILTERING.

5.2. Pregunta 13. Dos consultas adicionales no realizables con CQL

Pregunta 13. Añade a `pivot.sc` dos consultas más que no se puedan realizar con CQL.

Además de las consultas anteriores, se añadieron al fichero `pivot.sc` **dos consultas adicionales** utilizando Spark SQL que no pueden resolverse directamente con CQL, debido a las limitaciones de Cassandra en operaciones globales de agregación, agrupación y ordenación. Partiendo de la vista temporal ya creada en la pregunta anterior, se incorporaron las siguientes consultas:

```
// Consulta adicional 1:
// Personas con mayor número de pruebas registradas
val sql_top_personas = spark.sql("""SELECT person, COUNT(*) AS total_pruebas FROM lab
GROUP BY person ORDER BY total_pruebas DESC LIMIT 10""").show()

// Consulta adicional 2:
// Media observada por tipo de prueba, filtrando las de media >= 100
val sql_media_por_prueba = spark.sql("""SELECT std_observable_cd, AVG(obs_value_nm) AS
media_prueba FROM lab GROUP BY std_observable_cd HAVING AVG(obs_value_nm) >= 100 ORDER
BY media_prueba DESC""").show()
```

La **primera consulta** agrupa las filas por persona, cuenta cuántas pruebas tiene registrada cada una y ordena el resultado de mayor a menor, mostrando únicamente las diez primeras. De este modo, permite identificar las personas con mayor número de pruebas almacenadas en la tabla. La salida obtenida fue la siguiente:

```
+-----+-----+
|person|total_pruebas|
+-----+-----+
|271041|          25|
|169820|          25|
|  3691|          25|
| 30720|          25|
|  2094|          25|
| 62123|          25|
|146550|          25|
|121092|          25|
```



```
| 3108 | 25 |
| 37474 | 25 |
+-----+
sql_top_personas: Unit = ()
```

La **segunda consulta** agrupa los datos por código de prueba, calcula la media del valor observado (obs_value_nm) para cada una, filtra únicamente aquellas pruebas cuya media es mayor o igual que 100 y, finalmente, ordena el resultado de mayor a menor según esa media. La salida obtenida fue:

```
+-----+-----+
|std_observable_cd| media_prueba|
+-----+-----+
|          3255-7| 590.4074615851787|
|          26515-7| 256.57081942090457|
|           2951-2| 140.8972308056898|
|          41652-9| 135.06627315297135|
+-----+-----+
sql_media_por_prueba: Unit = ()
```

Estas dos consultas **no pueden realizarse directamente en CQL** porque requieren operaciones de agrupación global (GROUP BY), filtrado sobre resultados agregados (HAVING) y ordenación del resultado (ORDER BY) en función de valores calculados. En cambio, Spark SQL sí permite expresar este tipo de análisis de forma natural sobre la vista temporal **lab**.

6. Dificultades encontradas y lecciones aprendidas

6.1. Problema con la imagen de Spark en el fichero docker-compose.yml

Durante la fase de preparación del entorno de trabajo se detectó una incidencia en el fichero [docker-compose.yml](#) proporcionado para la práctica. En concreto, la configuración original utilizaba la imagen **bitnami/spark:3.3** para el **nodo maestro** de Spark y para los **dos workers**. Sin embargo, al ejecutar **docker-compose up**, Docker no pudo resolver dicha referencia y devolvió un error indicando que la imagen no se encontraba disponible.

```
root@core ~/bd2-practica5# docker-compose up
WARN[0000] /root/bd2-practica5/docker-compose.yml: the attribute `version` is obsolete,
it will be ignored, please remove it to avoid potential confusion
[+] up 1/1
! Image docker.io/bitnami/spark:3.3 Interrupted
0.8s
Error response from daemon: failed to resolve reference "docker.io/bitnami/spark:3.3":
docker.io/bitnami/spark:3.3: not found
```

Antes de adoptar una modificación del fichero `docker-compose.yml`, se valoró la posibilidad de **sustituir** únicamente **el repositorio de la imagen** ([docker-compose.yml](#)) y mantener, en la medida de lo posible, la configuración original basada en Bitnami. En concreto, se consideró el uso del repositorio **docker.io/bitnamilegacy**, creado por Bitnami como archivo de imágenes antiguas tras los cambios introducidos en su catálogo público. Esta alternativa resultaba **atractiva** porque podía permitir conservar parte de la lógica original del despliegue. No obstante, esta **opción fue descartada** como solución final.

Por un lado, el repositorio legacy se presenta como una **solución de transición**, sin mantenimiento ni soporte futuro. Por otro, existen incidencias públicas que muestran que dicha alternativa **no garantiza** de forma completa **la estabilidad ni la disponibilidad** de todos los tags antiguos. Por ello, aunque bitnamilegacy podía considerarse una medida de contingencia menos invasiva, **no se estimó suficientemente robusta** para sustentar la configuración del entorno.

En consecuencia, se optó por una **solución más sólida y reproducible**: sustituir la imagen de Bitnami por una **imagen oficial de Apache Spark**, manteniendo la misma arquitectura general del despliegue, esto es, un nodo maestro, dos workers y el montaje del directorio de trabajo en `/data`. Esta **primera adaptación** permitió poner en funcionamiento el entorno y continuar con las comprobaciones iniciales. Sin embargo, más adelante se comprobó que dicha sustitución no resolvía completamente el problema, ya que introducía una **incompatibilidad** entre la **versión de Spark** utilizada y el **conector de Cassandra** requerido por la práctica.

En efecto, una vez conseguido el **arranque del shell de Spark**, se observó que el entorno se estaba ejecutando con una combinación de **Spark y Scala** no compatible con el conector **spark-cassandra-connector_2.12:3.3.0** indicado en el enunciado. Como consecuencia, fue necesario realizar una **segunda adaptación** del fichero `docker-compose.yml`, seleccionando una **imagen de Spark** más próxima a la versión esperada por el conector y compatible con **Scala 2.12**. La imagen finalmente adoptada fue la siguiente:

```
apache/spark:3.3.3-scala2.12-java11-r-ubuntu
```

Esta modificación se aplicó a los **tres servicios de Spark** (`spark`, `spark-worker-1` y `spark-worker-2`), manteniéndose sin cambios el servicio de Cassandra y el montaje del directorio local en `/data`. Una vez actualizado el fichero de configuración, se procedió a **desplegar de nuevo** el entorno. Para ello, se siguieron los siguientes pasos:

1. Parada y limpieza del despliegue anterior.

```
root@core ~/bd2-practica5# docker-compose down --remove-orphans
[+] down 5/5
✓ Container spark-worker-2      Removed      1.9s
✓ Container spark-worker-1      Removed      1.8s
✓ Container spark               Removed      1.4s
✓ Container cassandra           Removed      7.0s
✓ Network bd2-practica5_default Removed      0.2s
```

2. Descarga de la nueva imagen.

```
root@core ~/bd2-practica5# docker-compose pull
[+] pull 13/13
✓ Image cassandra:4.1 Pulled 1.7sss
✓ Image apache/spark:3.3.3-scala2.12-java11-r-ubuntu Pulled 100.7s
```

3. Arranque del entorno actualizado.

```
root@core ~/bd2-practica5# docker-compose up -d
[+] up 5/5
✓ Network bd2-practica5_default Created 0.4s
✓ Container cassandra Created 1.4s
✓ Container spark Created 0.2s
✓ Container spark-worker-1 Created 0.3s
✓ Container spark-worker-2 Created 0.3s
```

4. Comprobación del estado de los servicios.

```
root@core ~/bd2-practica5# docker compose ps
```

NAME	IMAGE	STATUS	PORTS	COMMAND
SERVICE	CREATED			
cassandra	cassandra:4.1			"docker-entrypoint.s..."
cassandra	24 seconds ago	Up 22 seconds	7000-7001/tcp, 7199/tcp, 9160/tcp, 0.0.0.0:9042->9042/tcp, [::]:9042->9042/tcp	
spark	apache/spark:3.3.3-scala2.12-java11-r-ubuntu			"/opt/entrypoint.sh ..."
spark	24 seconds ago	Up 21 seconds	0.0.0.0:7077->7077/tcp, [::]:7077->7077/tcp, 0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp	
spark-worker-1	apache/spark:3.3.3-scala2.12-java11-r-ubuntu			"/opt/entrypoint.sh ..."
spark-worker-1	23 seconds ago	Up 21 seconds	0.0.0.0:8081->8081/tcp, [::]:8081->8081/tcp	
spark-worker-2	apache/spark:3.3.3-scala2.12-java11-r-ubuntu			"/opt/entrypoint.sh ..."
spark-worker-2	23 seconds ago	Up 21 seconds	0.0.0.0:8082->8082/tcp, [::]:8082->8082/tcp	

La salida obtenida confirmó que los **cuatro contenedores** se encontraban correctamente en ejecución: cassandra, spark, spark-worker-1 y spark-worker-2. Asimismo, se verificó que el nodo maestro de Spark y sus dos workers quedaban expuestos en los puertos esperados.

6.2. Incidencia en el acceso a Spark y resolución de dependencias de Cassandra

Una vez resuelto el problema inicial de despliegue del entorno, surgieron nuevas dificultades al intentar acceder a Spark y utilizar el conector de Cassandra necesario para la práctica. Aunque el enunciado propone arrancar Spark mediante un comando directo con **spark-shell** y la **opción --packages**, en el entorno finalmente configurado este procedimiento no funcionó de forma inmediata. En una **primera prueba** se intentó acceder a Spark con el comando indicado de forma aproximada en el enunciado. Sin embargo, el sistema devolvió un error indicando que el ejecutable spark-shell no se encontraba accesible en el **PATH del contenedor**.

```
root@core ~/bd2-practica5# docker-compose exec spark spark-shell --packages
com.datastax.spark:spark-cassandra-connector_2.12:3.3.0
OCI runtime exec failed: exec failed: unable to start container process: exec:
"spark-shell": executable file not found in $PATH
```

Para analizar la situación, **se accedió manualmente** al contenedor y se inspeccionó el contenido del directorio **/opt/spark/bin**, comprobándose que **el binario sí existía**, pero no podía invocarse directamente como comando simple.

```
root@core ~/bd2-practica5# docker-compose exec spark bash

spark@spark:/opt/spark/work-dir$ ls /opt/spark/bin
total 120
-rwxr-xr-x 1 spark spark 1089 Feb  2 08:36 beeline
-rw-r--r-- 1 spark spark 1064 Feb  2 08:36 beeline.cmd
-rwxr-xr-x 1 spark spark 10983 Feb  2 08:36 docker-image-tool.sh
-rwxr-xr-x 1 spark spark 1935 Feb  2 08:36 find-spark-home
-rw-r--r-- 1 spark spark 2685 Feb  2 08:36 find-spark-home.cmd
-rw-r--r-- 1 spark spark 2421 Feb  2 08:36 load-spark-env.cmd
-rw-r--r-- 1 spark spark 2741 Feb  2 08:36 load-spark-env.sh
-rwxr-xr-x 1 spark spark 4051 Feb  2 08:36 pyspark
-rw-r--r-- 1 spark spark 1542 Feb  2 08:36 pyspark2.cmd
-rw-r--r-- 1 spark spark 1170 Feb  2 08:36 pyspark.cmd
-rwxr-xr-x 1 spark spark 1030 Feb  2 08:36 run-example
-rw-r--r-- 1 spark spark 1223 Feb  2 08:36 run-example.cmd
-rwxr-xr-x 1 spark spark 3561 Feb  2 08:36 spark-class
-rwxr-xr-x 1 spark spark 3536 Feb  2 08:36 spark-class2.cmd
-rw-r--r-- 1 spark spark 1180 Feb  2 08:36 spark-class.cmd
-rwxr-xr-x 1 spark spark 1067 Feb  2 08:36 spark-connect-shell
-rwxr-xr-x 1 spark spark 1075 Feb  2 08:36 sparkR
-rw-r--r-- 1 spark spark 1125 Feb  2 08:36 sparkR2.cmd
-rw-r--r-- 1 spark spark 1168 Feb  2 08:36 sparkR.cmd
-rwxr-xr-x 1 spark spark 3283 Feb  2 08:36 spark-shell
-rw-r--r-- 1 spark spark 1818 Feb  2 08:36 spark-shell2.cmd
-rw-r--r-- 1 spark spark 1178 Feb  2 08:36 spark-shell.cmd
-rwxr-xr-x 1 spark spark 1093 Feb  2 08:36 spark-sql
-rw-r--r-- 1 spark spark 1146 Feb  2 08:36 spark-sql2.cmd
-rw-r--r-- 1 spark spark 1173 Feb  2 08:36 spark-sql.cmd
-rwxr-xr-x 1 spark spark 1040 Feb  2 08:36 spark-submit
-rw-r--r-- 1 spark spark 1155 Feb  2 08:36 spark-submit2.cmd
-rw-r--r-- 1 spark spark 1180 Feb  2 08:36 spark-submit.cmd

spark@spark:/opt/spark/work-dir$ ls /opt/spark/bin/spark-shell
/opt/spark/bin/spark-shell
```

A partir de esta comprobación, se decidió lanzar el shell utilizando su ruta completa:

```
/opt/spark/bin/spark-shell
```

Tras este primer problema, apareció una segunda incidencia al intentar descargar el conector de Cassandra mediante la **opción --packages**. En este caso, Spark utilizaba **Ivy** para resolver **dependencias Maven**, pero intentaba escribir en una ruta no válida asociada al

directorio personal del usuario del contenedor (/nonexistent/.ivy...), lo que provocaba una excepción de tipo **FileNotFoundException**.

```
root@core ~/bd2-practica5# docker-compose exec spark /opt/spark/bin/spark-shell --master
spark://spark:7077 --packages com.datastax.spark:spark-cassandra-connector_2.12:3.3.0
WARNING: Using incubator modules: jdk.incubator.vector
:: loading settings :: url =
jar:file:/opt/spark/jars/ivy-2.5.3.jar!/org/apache/ivy/core/settings/ivysettings.xml
Ivy Default Cache set to: /nonexistent/.ivy2.5.2/cache
The jars for the packages stored in: /nonexistent/.ivy2.5.2/jars
com.datastax.spark#spark-cassandra-connector_2.12 added as a dependency
:: resolving dependencies ::
org.apache.spark#spark-submit-parent-f16850af-1554-4672-8af8-e620ebdaf5b8;1.0
  confs: [default]
Exception in thread "main" java.io.FileNotFoundException:
/nonexistent/.ivy2.5.2/cache/resolved-org.apache.spark-spark-submit-parent-f16850af-1554-4672-8af8-e620ebdaf5b8-1.0.xml (No such file or directory)
  at java.base/java.io.FileOutputStream.open0(Native Method)
  at java.base/java.io.FileOutputStream.open(FileOutputStream.java:293)
  at java.base/java.io.FileOutputStream.<init>(FileOutputStream.java:235)
  at java.base/java.io.FileOutputStream.<init>(FileOutputStream.java:184)
  at
org.apache.ivy.plugins.parser.xml.XmlModuleDescriptorWriter.write(XmlModuleDescriptorWriter.java:71)
  at
org.apache.ivy.plugins.parser.xml.XmlModuleDescriptorWriter.write(XmlModuleDescriptorWriter.java:63)
  at
org.apache.ivy.core.module.descriptor.DefaultModuleDescriptor.toIvyFile(DefaultModuleDescriptor.java:553)
  at
org.apache.ivy.core.cache.DefaultResolutionCacheManager.saveResolvedModuleDescriptor(DefaultResolutionCacheManager.java:184)
    at org.apache.ivy.core.resolve.ResolveEngine.resolve(ResolveEngine.java:259)
    at org.apache.ivy.Ivy.resolve(Ivy.java:522)
    at
org.apache.spark.util.MavenUtils$.resolveMavenCoordinates(MavenUtils.scala:511)
  at
org.apache.spark.util.DependencyUtils$.resolveMavenDependencies(DependencyUtils.scala:123)
    at
org.apache.spark.deploy.SparkSubmit.prepareSubmitEnvironment(SparkSubmit.scala:341)
  at
org.apache.spark.deploy.SparkSubmit.org$apache$spark$deploy$SparkSubmit$$runMain(SparkSubmit.scala:961)
    at org.apache.spark.deploy.SparkSubmit.doRunMain$1(SparkSubmit.scala:204)
    at org.apache.spark.deploy.SparkSubmit.submit(SparkSubmit.scala:227)
    at org.apache.spark.deploy.SparkSubmit.doSubmit(SparkSubmit.scala:96)
    at org.apache.spark.deploy.SparkSubmit$$anon$2.doSubmit(SparkSubmit.scala:1132)
    at org.apache.spark.deploy.SparkSubmit$.main(SparkSubmit.scala:1141)
    at org.apache.spark.deploy.SparkSubmit.main(SparkSubmit.scala)
```

Para resolverlo, se creó **manualmente** un **directorio temporal** dentro del contenedor.

```
docker-compose exec spark bash -c "mkdir -p /tmp/.ivy2"
```

Y se volvió a lanzar Spark indicando **explícitamente** dicha ruta como directorio de trabajo para Ivy.

```
docker-compose exec spark /opt/spark/bin/spark-shell \  
  --master spark://spark:7077 \  
  --conf spark.jars.ivy=/tmp/.ivy2 \  
  --packages com.datastax.spark:spark-cassandra-connector_2.12:3.3.0
```

Aun así, durante las pruebas se detectó una tercera incidencia: la combinación inicialmente utilizada de Spark y Scala no era compatible con el conector **spark-cassandra-connector_2.12:3.3.0**. Este problema surgió al cargar el script **pivot.sc**, cuando Spark lanzó una excepción **NoSuchMethodError**, señal de una **incompatibilidad binaria** entre el entorno de ejecución y las librerías del conector. El origen del problema era la diferencia entre la **versión de Scala** empleada por Spark en esa configuración intermedia y la **versión** para la que había sido compilado el conector.

La **resolución** de esta incidencia se consiguió mediante la **modificación** descrita en el **apartado anterior**, sustituyendo la imagen de Spark por una variante compatible con Scala 2.12. Una vez desplegado de nuevo el entorno con **apache/spark:3.3.3-scala2.12-java11-r-ubuntu**, el acceso a Spark pudo realizarse correctamente. Con esta configuración, Spark resolvió y descargó correctamente todas las **dependencias** requeridas, inició el **shell interactivo** sin errores y permitió cargar el script **pivot.sc** desde **/data/pivot.sc**. Posteriormente, al ejecutar:

```
:load /data/pivot.sc
```

Se comprobó que Spark podía acceder al keyspace **laboratorio** y crear correctamente el DataFrame **laboratory_table**.

En conjunto, esta incidencia mostró que, tras modificar la imagen de Spark para resolver el problema inicial de despliegue, también fue necesario adaptar el modo de acceso al shell y la configuración de resolución de dependencias. En particular, la solución final requirió:

- utilizar la ruta completa del ejecutable spark-shell;
- indicar explícitamente el nodo maestro de Spark;
- definir una ruta válida para la caché de Ivy;
- y emplear una versión de Spark compatible con el conector de Cassandra especificado en el enunciado.

De este modo, el entorno quedó finalmente preparado para desarrollar la parte de explotación analítica con Spark.

7. Conclusiones e impresiones personales

La práctica ha permitido abordar **de forma aplicada** el uso de **Cassandra** como SGBD NoSQL y su explotación posterior mediante **Spark**. A lo largo del trabajo se ha comprobado que Cassandra presenta un **modelo de funcionamiento distinto** al de los SGBD relacionales de prácticas anteriores, tanto en la organización de los datos como en la forma de plantear las consultas.

La **carga del conjunto de datos** ha permitido observar algunas características del modelo de Cassandra. Por un lado, se ha comprobado que el **número de filas procesadas** durante la importación no coincide necesariamente con el **número de filas finalmente almacenadas**, debido al diseño de la clave primaria y a la política Last-Write-Wins, que provoca la sobrescritura de registros cuando varias observaciones comparten la misma clave. Por otro, se ha constatado que **operaciones aparentemente sencillas** en un entorno relacional, como COUNT(*) o ciertas consultas con DISTINCT, pueden resultar **inviabiles** en Cassandra cuando el volumen de datos es elevado.

En este contexto, **Spark** ha mostrado su utilidad como **complemento**. Frente a las limitaciones de CQL, el **uso de DataFrames y de Spark SQL** ha permitido resolver consultas de filtrado, agregación, pivotado y ordenación que en Cassandra no podían expresarse de forma eficiente. De este modo, la práctica ha evidenciado una idea central en Big Data: Cassandra resulta adecuada para el **almacenamiento distribuido** y el **acceso eficiente por clave**, mientras que Spark destaca como **herramienta de análisis** sobre grandes volúmenes de datos.

Desde un punto de vista práctico, el desarrollo de la práctica también ha puesto de manifiesto la necesidad de **adaptar cuidadosamente el entorno de trabajo**. La sustitución de imágenes Docker, los ajustes de compatibilidad entre Spark, Scala y el conector de Cassandra, así como los **problemas de recursos** observados durante la carga masiva del CSV, han servido para reforzar la importancia de documentar versiones, verificar dependencias y comprender el comportamiento de las herramientas utilizadas.

En conjunto, la práctica ha resultado útil para comprender las **diferencias entre el enfoque relacional tradicional y el enfoque distribuido** orientado a grandes volúmenes de datos. Además, ha permitido adquirir una visión más realista del trabajo con **tecnologías NoSQL**, desde su despliegue, explotación y análisis en un entorno técnico real.

8. Distribución de trabajo

Apartados del trabajo realizados	Lucía	Luna
1. Configuración inicial		
Preparación del entorno Docker y resolución de problemas de red/DNS	0,5	0,5
Análisis preliminar del dataset CSV y estructura de datos	0,2	0,2
2. Cassandra y CQL (fichero laboratorio.cql)		
Creación de tablas y carga masiva de datos (COPY)	0,5	0,5
Desarrollo de consultas analíticas y filtrados en CQL (DISTINCT, ALLOW FILTERING)	0,2	0,2
Análisis de errores y limitaciones de Cassandra (Timeouts, recuentos globales)	1	1
Investigación sobre arquitectura NoSQL (Keyspace, política Last-Write-Wins)	1	1
3. Apache Spark: API DataFrames (fichero pivot.sc)		
Conexión Spark-Cassandra y operación de pivotado de datos	0,5	0,5
Resolución de consultas analíticas usando la API de DataFrames (filter, select, avg)	1	1
4. Apache Spark: Spark SQL (fichero pivot.sc)		
Configuración de vistas temporales y traducción de consultas a SQL puro	0,5	0,5
Diseño de consultas avanzadas incompatibles con CQL (GROUP BY, HAVING)	0,5	0,5
5. Documentación y memoria		
Redacción de la memoria	1,5	1,5
	7,4	7,4

9. Bibliografía

Apache Spark. (s. f.). Configuration. Spark Documentation.
<https://spark.apache.org/docs/latest/configuration.html>

Apache Spark. (s. f.). Downloads. <https://spark.apache.org/downloads.html>

Apache Spark. (s. f.). Spark Standalone Mode. Spark Documentation.
<https://spark.apache.org/docs/latest/spark-standalone.html>

bhbyuh. (2026, 4 de febrero). bitnami/spark image showing no tags at Docker Hub (Issue #90528). GitHub. <https://github.com/bitnami/containers/issues/90528>

Docker, Inc. (s. f.). Control startup order. Docker Docs.
<https://docs.docker.com/compose/how-tos/startup-order/>

Docker, Inc. (s. f.). Services. Docker Docs.
<https://docs.docker.com/reference/compose-file/services/>

Docker Hub. (s. f.). apache/spark. <https://hub.docker.com/r/apache/spark/>

Docker Hub. (s. f.). bitnami/spark tags. <https://hub.docker.com/r/bitnami/spark/tags>

Docker Hub. (s. f.). Image layer details: apache/spark:3.3.3-scala2.12-java11-r-ubuntu.
<https://hub.docker.com/layers/apache/spark/3.3.3-scala2.12-java11-r-ubuntu/images/sha256-a25bcf18d8bfba44afc60f1b62f06fceddec24c9bc5ccfc0d97e06cf199c4c02>

MvnRepository. (s. f.). Maven Repository: com.datastax.spark » spark-cassandra-connector_2.12 » 3.3.0 – Changes.
https://mvnrepository.com/artifact/com.datastax.spark/spark-cassandra-connector_2.12/3.3.0/changes

RaphHaddad. (2025, 3 de octubre). Unable to pull bitnami/spark container image from docker registry (Issue #13131). GitHub.
<https://github.com/actions/runner-images/issues/13131>

Sonatype. (s. f.). Maven Central Repository: com.datastax.spark:spark-cassandra-connector_2.12:3.3.0.
https://central.sonatype.com/artifact/com.datastax.spark/spark-cassandra-connector_2.12/3.3.0

wjimenez5271. (2025, 16 de julio). Upcoming changes to the Bitnami catalog (effective August 28th, 2025) (Issue #83267). GitHub.
<https://github.com/bitnami/containers/issues/83267>

10. Anexos

10.1. Anexo I. Archivos de configuración y scripts

10.1.1. docker-compose.yml (ORIGINAL)

```
version: '2'

services:
  cassandra:
    image: cassandra
    ports:
      - 9042:9042
    environment:
      - CASSANDRA_CLUSTER_NAME=cloudinfra
    volumes:
      - "./data:rw"
  spark:
    image: docker.io/bitnami/spark:3.3
    environment:
      - SPARK_MODE=master
      - SPARK_RPC_AUTHENTICATION_ENABLED=no
      - SPARK_RPC_ENCRYPTION_ENABLED=no
      - SPARK_LOCAL_STORAGE_ENCRYPTION_ENABLED=no
      - SPARK_SSL_ENABLED=no
      - SPARK_USER=spark
    ports:
      - '8080:8080'
    volumes:
      - "./data:rw"
  spark-worker-1:
    image: docker.io/bitnami/spark:3.3
    environment:
      - SPARK_MODE=worker
      - SPARK_MASTER_URL=spark://spark:7077
      - SPARK_WORKER_MEMORY=1G
      - SPARK_WORKER_CORES=1
      - SPARK_RPC_AUTHENTICATION_ENABLED=no
      - SPARK_RPC_ENCRYPTION_ENABLED=no
      - SPARK_LOCAL_STORAGE_ENCRYPTION_ENABLED=no
      - SPARK_SSL_ENABLED=no
      - SPARK_USER=spark
    volumes:
      - "./data:rw"
  spark-worker-2:
    image: docker.io/bitnami/spark:3.3
    environment:
      - SPARK_MODE=worker
      - SPARK_MASTER_URL=spark://spark:7077
      - SPARK_WORKER_MEMORY=1G
      - SPARK_WORKER_CORES=1
      - SPARK_RPC_AUTHENTICATION_ENABLED=no
      - SPARK_RPC_ENCRYPTION_ENABLED=no
      - SPARK_LOCAL_STORAGE_ENCRYPTION_ENABLED=no
      - SPARK_SSL_ENABLED=no
      - SPARK_USER=spark
    volumes:
      - "./data:rw"
```

10.1.2. docker-compose.yml (ADAPTADO con imágenes Docker actualizadas)

```
services:
  cassandra:
    image: cassandra:4.1
    container_name: cassandra
    ports:
      - "9042:9042"
    environment:
      - CASSANDRA_CLUSTER_NAME=cloudinfra
    volumes:
      - "./data:rw"

  spark:
    image: apache/spark:3.3.3-scala2.12-java11-r-ubuntu
    container_name: spark
    hostname: spark
    command:
      [
        "/opt/spark/bin/spark-class",
        "org.apache.spark.deploy.master.Master",
        "--host",
        "spark",
        "--port",
        "7077",
        "--webui-port",
        "8080"
      ]
    ports:
      - "7077:7077"
      - "8080:8080"
    volumes:
      - "./data:rw"
    depends_on:
      - cassandra

  spark-worker-1:
    image: apache/spark:3.3.3-scala2.12-java11-r-ubuntu
    container_name: spark-worker-1
    command:
      [
        "/opt/spark/bin/spark-class",
        "org.apache.spark.deploy.worker.Worker",
        "spark://spark:7077",
        "--webui-port",
        "8081"
      ]
    environment:
      - SPARK_WORKER_CORES=1
      - SPARK_WORKER_MEMORY=1G
    ports:
      - "8081:8081"
    volumes:
      - "./data:rw"
    depends_on:
      - spark

  spark-worker-2:
    image: apache/spark:3.3.3-scala2.12-java11-r-ubuntu
    container_name: spark-worker-2
    command:
```

```

[
  "/opt/spark/bin/spark-class",
  "org.apache.spark.deploy.worker.Worker",
  "spark://spark:7077",
  "--webui-port",
  "8082"
]
environment:
  - SPARK_WORKER_CORES=1
  - SPARK_WORKER_MEMORY=1G
ports:
  - "8082:8082"
volumes:
  - "./data:rw"
depends_on:
  - spark

```

10.1.3. docker-compose.yml (ADAPTADO con bitnamilegacy)

```

version: '2'

services:
  cassandra:
    image: cassandra
    ports:
      - 9042:9042
    environment:
      - CASSANDRA_CLUSTER_NAME=cloudinfra
    volumes:
      - "./data:rw"
  spark:
    image: docker.io/bitnamilegacy/spark:3.3
    environment:
      - SPARK_MODE=master
      - SPARK_RPC_AUTHENTICATION_ENABLED=no
      - SPARK_RPC_ENCRYPTION_ENABLED=no
      - SPARK_LOCAL_STORAGE_ENCRYPTION_ENABLED=no
      - SPARK_SSL_ENABLED=no
      - SPARK_USER=spark
    ports:
      - '8080:8080'
    volumes:
      - "./data:rw"
  spark-worker-1:
    image: docker.io/bitnamilegacy/spark:3.3
    environment:
      - SPARK_MODE=worker
      - SPARK_MASTER_URL=spark://spark:7077
      - SPARK_WORKER_MEMORY=1G
      - SPARK_WORKER_CORES=1
      - SPARK_RPC_AUTHENTICATION_ENABLED=no
      - SPARK_RPC_ENCRYPTION_ENABLED=no
      - SPARK_LOCAL_STORAGE_ENCRYPTION_ENABLED=no
      - SPARK_SSL_ENABLED=no
      - SPARK_USER=spark
    volumes:
      - "./data:rw"
  spark-worker-2:
    image: docker.io/bitnamilegacy/spark:3.3
    environment:

```

```

- SPARK_MODE=worker
- SPARK_MASTER_URL=spark://spark:7077
- SPARK_WORKER_MEMORY=1G
- SPARK_WORKER_CORES=1
- SPARK_RPC_AUTHENTICATION_ENABLED=no
- SPARK_RPC_ENCRYPTION_ENABLED=no
- SPARK_LOCAL_STORAGE_ENCRYPTION_ENABLED=no
- SPARK_SSL_ENABLED=no
- SPARK_USER=spark
volumes:
- "./data:rw"

```

10.1.4. laboratorio.cql

```

CREATE KEYSPACE IF NOT EXISTS laboratorio WITH REPLICATION =
{ 'class': 'SimpleStrategy',
  'replication_factor': '1'
};

CREATE TABLE laboratorio.hospital (
  std_observable_cd text,
  person int,
  std_observable_st text,
  obs_value_nm float,
  PRIMARY KEY ((std_observable_cd), person)
) WITH CLUSTERING ORDER BY (person ASC);

// carga de datos con COPY
COPY laboratorio.hospital (person, std_observable_cd, std_observable_st, obs_value_nm)
FROM '/data/dataset.csv'
WITH HEADER = TRUE AND DELIMITER = ';';

// --- Pregunta 4 ---

println("Ejecutando consulta de la Pregunta 4...")

SELECT DISTINCT std_observable_cd FROM laboratorio.hospital;

// -- Pregunta 7 ---

println("Ejecutando consultas de la Pregunta 7...")

SELECT AVG(obs_value_nm) FROM laboratorio.hospital WHERE std_observable_cd = '3255-7';

SELECT AVG(obs_value_nm) FROM laboratorio.hospital WHERE person = 123498 ALLOW
FILTERING;

SELECT * FROM laboratorio.hospital WHERE obs_value_nm >= 100 ALLOW FILTERING;

```

10.1.5. pivot.sc

```

import org.apache.spark.sql.functions._
// Se define el valor cassandraProperties: un diccionario

```

```
// con los datos para conectarse al keyspace laboratorio
// de la máquina cassandra
val cassandraProperties = Map("spark.cassandra.connection.host" -> "cassandra",
                              "spark.cassandra.connection.port" -> "9042",
                              "keyspace" -> "laboratorio")

// cargar la tabla hospital al valor laboratory_table
val laboratory_table =
  spark.read.format("org.apache.spark.sql.cassandra").
    options(cassandraProperties).
    option("table", "hospital").
    load()

// muestra las 20 primeras filas de la tabla
laboratory_table.show()

val pivotDF = laboratory_table.groupBy("person").pivot("std_observable_cd").sum("obs_value_nm")
pivotDF.show()
pivotDF.count

// --- Pregunta 9 ---
println("Ejecutando consultas de la Pregunta 9...")

val media_prueba_3255_7 = laboratory_table.filter("std_observable_cd = '3255-7'").select(
  avg("obs_value_nm")).show()

val media_persona_123498 = laboratory_table.filter("person = 123498").select(avg("obs_value_nm")).show()

val pruebas_mayor_100 = laboratory_table.filter("obs_value_nm >= 100").show()

// --- Pregunta 11 ---
println("Ejecutando consulta de la Pregunta 11...")

val media_prueba_1742_6 = laboratory_table.filter("std_observable_cd = '1742-6'").select(avg("obs_value_nm")).show()

// --- Pregunta 12 ---
println("Ejecutando consultas de la Pregunta 12...")

laboratory_table.createOrReplaceTempView("lab")

val consulta1 = spark.sql("SELECT AVG(obs_value_nm) FROM lab WHERE std_observable_cd = '3255-7'").show()

val consulta2 = spark.sql("SELECT AVG(obs_value_nm) FROM lab WHERE person = 123498").show()

val consulta3 = spark.sql("SELECT * FROM lab WHERE obs_value_nm >= 100").show()

val consulta4 = spark.sql("SELECT AVG(obs_value_nm) FROM lab WHERE std_observable_cd = '1742-6'").show()

// --- Pregunta 13 ---
println("Ejecutando consultas de la Pregunta 13...")

val sql_top_personas = spark.sql("SELECT person, COUNT(*) AS total_pruebas FROM lab GROUP BY person ORDER BY total_pruebas DESC LIMIT 10").show()

val sql_media_por_prueba = spark.sql("SELECT std_observable_cd, AVG(obs_value_nm) AS media_prueba FROM lab GROUP BY std_observable_cd HAVING AVG(obs_value_nm) >= 100 ORDER BY media_prueba DESC").show()
```

10.2. Anexo II. Salidas completas de ejecución y comprobación

10.2.1. Salida completa de ejecución de spark-shell con el conector de Cassandra

```

root@core ~/bd2-practica5# docker-compose exec spark /opt/spark/bin/spark-shell \
--master spark://spark:7077 \
--conf spark.jars.ivy=/tmp/.ivy2 \
--packages com.datastax.spark:spark-cassandra-connector_2.12:3.3.0
:: loading settings :: url =
jar:file:/opt/spark/jars/ivy-2.5.1.jar!/org/apache/ivy/core/settings/ivysettings.xml
Ivy Default Cache set to: /tmp/.ivy2/cache
The jars for the packages stored in: /tmp/.ivy2/jars
com.datastax.spark#spark-cassandra-connector_2.12 added as a dependency
:: resolving dependencies ::
org.apache.spark#spark-submit-parent-813a4e96-8d11-4841-99d1-991970a8b93c;1.0
  confs: [default]
  found com.datastax.spark#spark-cassandra-connector_2.12;3.3.0 in central
  found com.datastax.spark#spark-cassandra-connector-driver_2.12;3.3.0 in central
  found com.datastax.oss#java-driver-core-shaded;4.13.0 in central
  found com.datastax.oss#native-protocol;1.5.0 in central
  found com.datastax.oss#java-driver-shaded-guava;25.1-jre-graal-sub-1 in central
  found com.typesafe#config;1.4.1 in central
  found org.slf4j#slf4j-api;1.7.26 in central
  found io.dropwizard.metrics#metrics-core;4.1.18 in central
  found org.hdrhistogram#HdrHistogram;2.1.12 in central
  found org.reactivestreams#reactive-streams;1.0.3 in central
  found com.github.stephenc.jcip#jcip-annotations;1.0-1 in central
  found com.github.spotbugs#spotbugs-annotations;3.1.12 in central
  found com.google.code.findbugs#jsr305;3.0.2 in central
  found com.datastax.oss#java-driver-mapper-runtime;4.13.0 in central
  found com.datastax.oss#java-driver-query-builder;4.13.0 in central
  found org.apache.commons#commons-lang3;3.10 in central
  found com.thoughtworks.paranamer#paranamer;2.8 in central
  found org.scala-lang#scala-reflect;2.12.11 in central
downloading
https://repo1.maven.org/maven2/com/datastax/spark/spark-cassandra-connector_2.12/3.3.0/spark-cassandra-connector_2.12-3.3.0.jar ...
[SUCCESSFUL ]
com.datastax.spark#spark-cassandra-connector_2.12;3.3.0!spark-cassandra-connector_2.12.jar (292ms)
downloading
https://repo1.maven.org/maven2/com/datastax/spark/spark-cassandra-connector-driver_2.12/3.3.0/spark-cassandra-connector-driver_2.12-3.3.0.jar ...
[SUCCESSFUL ]
com.datastax.spark#spark-cassandra-connector-driver_2.12;3.3.0!spark-cassandra-connector-driver_2.12.jar (186ms)
downloading
https://repo1.maven.org/maven2/com/datastax/oss/java-driver-core-shaded/4.13.0/java-driver-core-shaded-4.13.0.jar ...
[SUCCESSFUL ] com.datastax.oss#java-driver-core-shaded;4.13.0!java-driver-core-shaded.jar (864ms)
downloading
https://repo1.maven.org/maven2/com/datastax/oss/java-driver-mapper-runtime/4.13.0/java-driver-mapper-runtime-4.13.0.jar ...
[SUCCESSFUL ]
com.datastax.oss#java-driver-mapper-runtime;4.13.0!java-driver-mapper-runtime.jar(bundle) (50ms)
downloading
https://repo1.maven.org/maven2/org/apache/commons/commons-lang3/3.10/commons-lang3-3.10.jar ...
[SUCCESSFUL ] org.apache.commons#commons-lang3;3.10!commons-lang3.jar (101ms)
downloading

```

```

https://repo1.maven.org/maven2/com/thoughtworks/paranamer/paranamer/2.8/paranamer-2.8.jar ...
[SUCCESSFUL ] com.thoughtworks.paranamer#paranamer;2.8!paranamer.jar(bundle) (50ms)
downloading
https://repo1.maven.org/maven2/org/scala-lang/scala-reflect/2.12.11/scala-reflect-2.12.11.jar ...
[SUCCESSFUL ] org.scala-lang#scala-reflect;2.12.11!scala-reflect.jar (494ms)
downloading
https://repo1.maven.org/maven2/com/datastax/oss/native-protocol/1.5.0/native-protocol-1.5.0.jar ...
[SUCCESSFUL ] com.datastax.oss#native-protocol;1.5.0!native-protocol.jar(bundle) (89ms)
downloading
https://repo1.maven.org/maven2/com/datastax/oss/java-driver-shaded-guava/25.1-jre-graal-sub-1/java-driver-shaded-guava-25.1-jre-graal-sub-1.jar ...
[SUCCESSFUL ]
com.datastax.oss#java-driver-shaded-guava;25.1-jre-graal-sub-1!java-driver-shaded-guava.jar (356ms)
downloading https://repo1.maven.org/maven2/com/typesafe/config/1.4.1/config-1.4.1.jar ...
[SUCCESSFUL ] com.typesafe#config;1.4.1!config.jar(bundle) (77ms)
downloading https://repo1.maven.org/maven2/org/slf4j/slf4j-api/1.7.26/slf4j-api-1.7.26.jar ...
[SUCCESSFUL ] org.slf4j#slf4j-api;1.7.26!slf4j-api.jar (57ms)
downloading
https://repo1.maven.org/maven2/io/dropwizard/metrics/metrics-core/4.1.18/metrics-core-4.1.18.jar ...
[SUCCESSFUL ] io.dropwizard.metrics#metrics-core;4.1.18!metrics-core.jar(bundle) (57ms)
downloading
https://repo1.maven.org/maven2/org/hdrhistogram/HdrHistogram/2.1.12/HdrHistogram-2.1.12.jar ...
[SUCCESSFUL ] org.hdrhistogram#HdrHistogram;2.1.12!HdrHistogram.jar(bundle) (63ms)
downloading
https://repo1.maven.org/maven2/org/reactivestreams/reactive-streams/1.0.3/reactive-streams-1.0.3.jar ...
[SUCCESSFUL ] org.reactivestreams#reactive-streams;1.0.3!reactive-streams.jar (47ms)
downloading
https://repo1.maven.org/maven2/com/github/stephenc/jcip/jcip-annotations/1.0-1/jcip-annotations-1.0-1.jar ...
[SUCCESSFUL ] com.github.stephenc.jcip#jcip-annotations;1.0-1!jcip-annotations.jar (44ms)
downloading
https://repo1.maven.org/maven2/com/github/spotbugs/spotbugs-annotations/3.1.12/spotbugs-annotations-3.1.12.jar ...
[SUCCESSFUL ] com.github.spotbugs#spotbugs-annotations;3.1.12!spotbugs-annotations.jar (50ms)
downloading https://repo1.maven.org/maven2/com/google/code/findbugs/jsr305/3.0.2/jsr305-3.0.2.jar ...
[SUCCESSFUL ] com.google.code.findbugs#jsr305;3.0.2!jsr305.jar (45ms)
downloading
https://repo1.maven.org/maven2/com/datastax/oss/java-driver-query-builder/4.13.0/java-driver-query-builder-4.13.0.jar ...
[SUCCESSFUL ]
com.datastax.oss#java-driver-query-builder;4.13.0!java-driver-query-builder.jar(bundle) (72ms)
:: resolution report :: resolve 13605ms :: artifacts dl 3079ms
:: modules in use:
com.datastax.oss#java-driver-core-shaded;4.13.0 from central in [default]
com.datastax.oss#java-driver-mapper-runtime;4.13.0 from central in [default]
com.datastax.oss#java-driver-query-builder;4.13.0 from central in [default]
com.datastax.oss#java-driver-shaded-guava;25.1-jre-graal-sub-1 from central in [default]
com.datastax.oss#native-protocol;1.5.0 from central in [default]
com.datastax.spark#spark-cassandra-connector-driver_2.12;3.3.0 from central in [default]
com.datastax.spark#spark-cassandra-connector_2.12;3.3.0 from central in [default]
com.github.spotbugs#spotbugs-annotations;3.1.12 from central in [default]
com.github.stephenc.jcip#jcip-annotations;1.0-1 from central in [default]
com.google.code.findbugs#jsr305;3.0.2 from central in [default]
com.thoughtworks.paranamer#paranamer;2.8 from central in [default]
com.typesafe#config;1.4.1 from central in [default]
io.dropwizard.metrics#metrics-core;4.1.18 from central in [default]
org.apache.commons#commons-lang3;3.10 from central in [default]
org.hdrhistogram#HdrHistogram;2.1.12 from central in [default]
org.reactivestreams#reactive-streams;1.0.3 from central in [default]

```



```
org.scala-lang#scala-reflect;2.12.11 from central in [default]
org.slf4j#slf4j-api;1.7.26 from central in [default]
```

	modules	artifacts
conf	number search dwnlded evicted number dwnlded	
default	18 18 18 0 18 18	

```
:: retrieving :: org.apache.spark#spark-submit-parent-813a4e96-8d11-4841-99d1-991970a8b93c
   confs: [default]
    18 artifacts copied, 0 already retrieved (18066kB/173ms)
26/04/22 04:02:04 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform...
using builtin-java classes where applicable
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
Spark context Web UI available at http://spark:4040
Spark context available as 'sc' (master = spark:///spark:7077, app id = app-20260422040219-0000).
Spark session available as 'spark'.
Welcome to

 / _/_/ _ _ _/_/_/ 
_ \ V _ V _ `/_/_/'/_/
/_/_/. _/\_,_/_/_/_/_\ version 3.3.3
/_/ 

Using Scala version 2.12.15 (OpenJDK 64-Bit Server VM, Java 11.0.20)
Type in expressions to have them evaluated.
Type :help for more information.
```

scala>

10.2.2. Salida completa de carga del script pivot.sc en Spark

```
scala> :load /data/pivot.sc
Loading /data/pivot.sc...
cassandraProperties: scala.collection.immutable.Map[String,String] =
Map(spark.cassandra.connection.host -> cassandra, spark.cassandra.connection.port ->
9042, keyspace -> laboratorio)
laboratory_table: org.apache.spark.sql.DataFrame = [std_observable_cd: string, person:
int ... 2 more fields]
+-----+-----+-----+-----+
|std_observable_cd|person|obs_value_nm|std_observable_st|
+-----+-----+-----+-----+
|18124-0|11|57.58|CD3 cells/100 cel...|
|18124-0|17|57.58|CD3 cells/100 cel...|
|18124-0|20|71.93|CD3 cells/100 cel...|
|18124-0|21|57.58|CD3 cells/100 cel...|
|18124-0|51|71.93|CD3 cells/100 cel...|
|18124-0|65|71.93|CD3 cells/100 cel...|
|18124-0|81|73.15|CD3 cells/100 cel...|
|18124-0|121|80.47|CD3 cells/100 cel...|
|18124-0|122|80.47|CD3 cells/100 cel...|
|18124-0|168|80.47|CD3 cells/100 cel...|
|18124-0|213|74.53|CD3 cells/100 cel...|
|18124-0|221|75.87|CD3 cells/100 cel...|
|18124-0|229|73.15|CD3 cells/100 cel...|
|18124-0|298|80.47|CD3 cells/100 cel...
```

[illegible]

```

80.19999694824219|31.399999618530273| null|0.9800000190734863|
null|11.699999809265137| null|
| 833| 21.0| 32.0|0.47999998927116394| 2.809999942779541|
null|31.600000381469727| 0.800000011920929| 0.04224000126123428| 0.0|
3.569999933242798| 6.800000190734863| 0.5551999807357788| 0.2856000065803528| 467.0|
4.309999942779541|31.299999237060547| 35.70000076293945|137.39999389648438|
88.5| 27.0| null|1.0700000524520874| null|12.699999809265137|
null|
| 1088| 8.0| 13.0| null| null| null|
35.0|0.7599999904632568| 0.03440000116825104| 0.2020999938249588| 3.680000066757202|
null| 0.9933000206947327| 0.5375000238418579| 93.0| null|
32.70000076293945| 34.5| null| 94.9000015258789|
null| null| null| null| null| null|
| 1238| 10.0| 24.0| null| 0.5199999809265137| null|
42.0|1.0299999713897705| 0.10000000149011612|0.10000000149011612|
4.480000019073486|14.399999618530273| 2.700000047683716| 1.100000023841858| 354.0|
4.900000095367432|31.600000381469727| 34.0| 138.0|
93.69999694824219|32.900001525878906| 360.0|0.9800000190734863| null|
11.5| null|
| 1342| 93.0| 53.0| null| 0.7400000095367432|12.199999809265137|
44.29999923706055|0.9200000166893005| 0.10000000149011612| 0.6000000238418579|
4.610000133514404|11.699999809265137| 3.200000047683716| 1.2000000476837158| 399.0|
4.5| 33.0| 34.20000076293945| 141.0| 97.80000305175781|
34.70000076293945| 447.0|1.0199999809265137| null| 12.0|
null|
| 1580| null| 106.0| null| null|109.19999694824219|
46.70000076293945|1.2100000381469727| 0.0|0.10000000149011612|
5.150000095367432| 7.300000190734863| 0.8999999761581421| 0.5| null|
4.269999980926514| 32.5|35.400001525878906| 142.0|
91.9000015258789|29.700000762939453| 845.0|1.3200000524520874|
null|14.600000381469727| null|
| 1591| null| null| null| null| null|
40.5|0.8299999833106995| 0.0|0.10000000149011612| 4.710000038146973|
9.800000190734863| 1.5| 0.8999999761581421| null|
4.019999980926514|29.700000762939453| 34.5| 131.0|
86.0|26.899999618530273| 445.0|1.2999999523162842| null|14.899999618530273|
null|
| 1645| 450.0| 108.0| null|
1.2100000381469727|30.600000381469727|48.900001525878906| 1.100000023841858|
0.15729999542236328|0.22990000247955322| 5.050000190734863| null|
4.9004998207092285| 1.1495000123977661| 260.0|
4.599999904632568|32.599998474121094|33.099998474121094| 134.0|
98.4000015258789| 39.20000076293945| 914.0|
1.25|169.38800048828125|15.399999618530273|75.37000274658203|
| 1829| 38.0| 27.0|0.0599999865889549| null|
2.200000047683716|41.599998474121094| 1.059999942779541|
0.0|0.30000001192092896| 5.0| 8.399999618530273| 3.200000047683716|
0.800000011920929| 359.0| 4.400000095367432|24.100000381469727| 32.5|
142.0| 81.19999694824219|28.700000762939453| 435.0|0.9900000095367432|
null|11.699999809265137| null|
| 1959| 17.0| 30.0| 0.3499999940395355| 0.9399999976158142|
null|45.599998474121094|0.7699999809265137|0.006899999920278788| 0.0|
5.019999980926514| 6.900000095367432|0.28290000557899475|0.47609999775886536| 186.0|
4.139999866485596|28.299999237060547| 33.79999923706055|142.60000610351562|
83.80000305175781|29.100000381469727| null| 1.309999942779541|

```

```

null|15.600000381469727|          null|
| 2122| 40.0| 28.0|0.07000000029802322| 0.33000001311302185|
7.880000114440918|39.900001525878906|0.8100000023841858| 0.09350000321865082|
0.3824999928474426|          5.0|          null| 4.232999801635742|
0.4542999863624573| 301.0|          null|25.899999618530273|31.799999237060547|
null|          81.0|          null| null|          null|          null|
null|          null|
| 2142| 34.0| 46.0|0.38999998569488525| 1.590000033378601|
null|39.400001525878906|0.8799999952316284|0.032930001616477966|0.27305999398231506|
4.289999961853027| 4.099999904632568| 1.2652599811553955|0.42517000436782837| 257.0|
5.019999980926514| 36.20000076293945|34.900001525878906|
141.8000030517578|103.80000305175781|30.399999618530273| null|1.0299999713897705|
null|12.300000190734863|          null|
| 2366| 28.0| 33.0|          null| 0.5400000214576721|          null|
53.0|0.7699999809265137| 0.09309999644756317|0.11088000237941742|
5.539999961853027|13.300000190734863| 3.098900079727173| 0.8245999813079834| 294.0|
5.53000020980835|30.899999618530273| 33.79999923706055|139.60000610351562|
92.0999984741211|31.200000762939453| null|1.0099999904632568|          111.0|
12.0|          null|
| 2659| 12.0| 17.0|0.10999999940395355| 0.6499999761581421| 6.099999904632568|
42.0|0.9300000071525574| 0.10000000149011612|0.20000000298023224|          4.5|
7.5| 2.200000047683716| 0.4000000059604645| 284.0|
4.599999904632568|30.700000762939453|33.599998474121094|          143.0|
93.4000015258789|27.200000762939453| 661.0|1.0700000524520874|
null|12.699999809265137|          null|
| 2866| 51.0| 29.0|          null|
1.2999999523162842|110.80000305175781|40.099998474121094|1.2999999523162842|
0.07699999958276749| 0.6478000283241272| 5.090000152587891|          null|
3.7070000171661377| 0.7590000033378601| 383.0| 4.199999809265137|30.200000762939453|
33.0|          143.0| 93.30000305175781|32.599998474121094| 645.0| 4.190000057220459|
182.0019989013672|51.900001525878906|          null|
| 3175| 42.0| 18.0|          null| 0.34999999940395355|
7.199999809265137|31.899999618530273| 1.340000033378601|
0.07450000196695328|0.31290000677108765|3.1700000762939453|          null|
4.306099891662598| 1.2963000535964966| 219.0|3.90000000953674316|          33.5|
33.29999923706055|          138.0| 100.5999984741211|31.299999237060547|
626.0|1.1200000047683716| 185.6060028076172|13.800000190734863|          null|
| 3749| 48.0| 39.0|          null| 0.4000000059604645|
null|38.900001525878906|0.6499999761581421| 0.09920000284910202|0.26767998933792114|
4.710000038146973|11.199999809265137| 1.9286400079727173| 0.9587200284004211| 554.0|
4.690000057220459|27.600000381469727|33.099998474121094|137.60000610351562|
84.9000015258789|          35.5| null|1.2100000381469727|
61.0|14.399999618530273|          null|
+-----+-----+-----+-----+-----+-----+-----+-----+
-----+-----+-----+-----+-----+-----+-----+-----+
--+-----+-----+-----+-----+-----+-----+-----+-----+
--+-----+-----+-----+-----+-----+-----+-----+-----+
-----+-----+-----+-----+-----+-----+-----+-----+-----+
--+

```

only showing top 20 rows

res2: Long = 306076

10.2.3. Salida completa de `laboratory_table.count`

```
scala> laboratory_table.count
26/04/22 09:32:17 ERROR Executor: Exception in task 3.0 in stage 21.0 (TID 31)6]
com.datastax.oss.driver.api.core.servererrors.ReadFailureException: Cassandra failure
during read query at consistency LOCAL_ONE (1 responses were required but only 0 replica
responded, 1 failed)
26/04/22 09:32:18 WARN TaskSetManager: Lost task 3.0 in stage 21.0 (TID 31)
(092ce4492dcb executor driver):
com.datastax.oss.driver.api.core.servererrors.ReadFailureException: Cassandra failure
during read query at consistency LOCAL_ONE (1 responses were required but only 0 replica
responded, 1 failed)

26/04/22 09:32:18 ERROR TaskSetManager: Task 3 in stage 21.0 failed 1 times; aborting
job
org.apache.spark.SparkException: Job aborted due to stage failure: Task 3 in stage 21.0
failed 1 times, most recent failure: Lost task 3.0 in stage 21.0 (TID 31) (092ce4492dcb
executor driver): com.datastax.oss.driver.api.core.servererrors.ReadFailureException:
Cassandra failure during read query at consistency LOCAL_ONE (1 responses were required
but only 0 replica responded, 1 failed)

Driver stacktrace:
    at
org.apache.spark.scheduler.DAGScheduler.failJobAndIndependentStages(DAGScheduler.scala:2
668)
    at
org.apache.spark.scheduler.DAGScheduler.$anonfun$abortStage$2(DAGScheduler.scala:2604)
    at
org.apache.spark.scheduler.DAGScheduler.$anonfun$abortStage$2$adapted(DAGScheduler.scala
:2603)
    at scala.collection.mutable.ResizableArray.foreach(ResizableArray.scala:62)
    at scala.collection.mutable.ResizableArray.foreach$(ResizableArray.scala:55)
    at scala.collection.mutable.ArrayBuffer.foreach(ArrayBuffer.scala:49)
    at org.apache.spark.scheduler.DAGScheduler.abortStage(DAGScheduler.scala:2603)
    at
org.apache.spark.scheduler.DAGScheduler.$anonfun$handleTaskSetFailed$1(DAGScheduler.sca
la:1178)
    at
org.apache.spark.scheduler.DAGScheduler.$anonfun$handleTaskSetFailed$1$adapted(DAGSchedu
ler.scala:1178)
    at scala.Option.foreach(Option.scala:407)
    at
org.apache.spark.scheduler.DAGScheduler.handleTaskSetFailed(DAGScheduler.scala:1178)
    at
org.apache.spark.scheduler.DAGSchedulerEventProcessLoop.doOnReceive(DAGScheduler.scala:2
856)
    at
org.apache.spark.scheduler.DAGSchedulerEventProcessLoop.onReceive(DAGScheduler.scala:279
8)
    at
org.apache.spark.scheduler.DAGSchedulerEventProcessLoop.onReceive(DAGScheduler.scala:278
7)
    at org.apache.spark.util.EventLoop$$anon$1.run(EventLoop.scala:49)
Caused by: com.datastax.oss.driver.api.core.servererrors.ReadFailureException: Cassandra
failure during read query at consistency LOCAL_ONE (1 responses were required but only 0
```

```
replica responded, 1 failed)
```

```
scala> 26/04/22 09:32:20 WARN TaskSetManager: Lost task 5.0 in stage 21.0 (TID 33)  
(092ce4492dcb executor driver): TaskKilled (Stage cancelled)
```

```
26/04/22 09:32:20 WARN TaskSetManager: Lost task 4.0 in stage 21.0 (TID 32)  
(092ce4492dcb executor driver): TaskKilled (Stage cancelled)
```